```
// AchievementFlow.cs
using System;
using System.Collections.Generic;
namespace Sy.Palace
{
    /// &lt;summary&gt;
    /// Pure view-model for the eight-achievement wall (task S2-T002 slice a,
    /// HallFlow paradigm): every trigger derives from the local sim event
    /// stream and the public sim counters - zero network faces (red line 12:
    /// offline progress, preset roster, nothing phones home), every unlock
    /// is deterministic and seed-independent (same forced trajectory = same
    /// unlock set on any seed). The eight medals per the taskbook row:
    /// first haul / first rank / zero-ad clear (U091 priority declaration -
    /// the full palace is reachable without a single rewarded ad) / album
    /// half / album full / seven consecutive court days / revenge ten /
    /// hundred-raid veteran. Unlocks are monotone (the wall only grows);
    /// Refresh is idempotent. The wall UI (programmatic fallback), the
    /// i18n titles (keys already registered) and the unlock telemetry beat
    /// (PalaceEventIds.AchievementUnlock) land with the wall host in slice b.
    /// &lt;/summary&gt;
    public class AchievementFlow
    {
        public const int MedalCount = 8;
        public const int CourtStreakDays = 7;
        public const int RevengeTenCount = 10;
        public const int VeteranRaidCount = 100;
        // medal ids (data-stable strings, PalaceEventIds paradigm)
        public const string FirstHaul = "ach_first_haul";
        public const string FirstRank = "ach_first_rank";
        public const string ZeroAdClear = "ach_zero_ad_clear";
        public const string AlbumHalf = "ach_album_half";
        public const string AlbumFull = "ach_album_full";
        public const string CourtWeek = "ach_court_week";
        public const string RevengeTen = "ach_revenge_ten";
        public const string VeteranHundred = "ach_veteran_hundred";
        public static readonly string[] All =
        {
            FirstHaul, FirstRank, ZeroAdClear, AlbumHalf, AlbumFull,
            CourtWeek, RevengeTen, VeteranHundred,
        };
        readonly PalaceSim _sim;
        readonly HashSet&lt;string&gt; _unlocked = new HashSet&lt;string&gt;();
        public AchievementFlow(PalaceSim sim)
        {
            if (sim == null) throw new ArgumentNullException("sim");
            _sim = sim;
        }
        public PalaceSim Sim { get { return _sim; } }
        public int UnlockedCount { get { return _unlocked.Count; } }
        public bool IsUnlocked(string id)
```

```
{
    return !string.IsNullOrEmpty(id) &amp;&amp; _unlocked.Contains(id);
}
/// &lt;summary&gt;Re-derive every trigger off the local event stream and
/// the sim counters. Idempotent and monotone: an unlocked medal
/// never un-locks (the wall only grows - local progress, red line 12).
/// Album medals consume the S3-T003 slice-a row-lit face (the same
/// law the album wall renders): the raw distinct-id count could hit
/// the full-album threshold with wall rows still dark, so the
/// reconciliation rides RowsLitOf, wall/achievement parity.&lt;/summary&gt;
public void Refresh()
{
    int albumRows = AlbumFlow.RowsLitOf(_sim.GalleryIds);
    if (HasHaulEvent()) Unlock(FirstHaul);
    if (_sim.CountEvents("rank_up") &gt; 0) Unlock(FirstRank);
    if (IsFullPalaceAdFree()) Unlock(ZeroAdClear);
    if (albumRows &gt;= G18Catalogs.AlbumItemCount / 2) Unlock(AlbumHalf);
    if (albumRows &gt;= G18Catalogs.AlbumItemCount) Unlock(AlbumFull);
    if (LongestCourtStreak() &gt;= CourtStreakDays) Unlock(CourtWeek);
    if (_sim.RevengeCount &gt;= RevengeTenCount) Unlock(RevengeTen);
    if (_sim.RaidCount &gt;= VeteranRaidCount) Unlock(VeteranHundred);
}
void Unlock(string id)
{
    _unlocked.Add(id);
}
bool HasHaulEvent()
{
    var events = _sim.Events;
    for (int i = 0; i &lt; events.Count; i++)
    {
        var e = events[i];
        if (e.Name == "raid_report" &amp;&amp; e.Detail != null &amp;&amp;
            e.Detail.StartsWith("verdict=haul", StringComparison.Ordinal)) return true;
    }
    return false;
}
/// &lt;summary&gt;Zero-ad clear (U091): the palace is complete (slot cap
/// reached, rank ladder topped) and not a single rewarded ad was ever
/// booked - the sim books ads only through rescue accepts and raid
/// doubles, so RescueCount + DoubleCount == total ads watched.&lt;/summary&gt;
bool IsFullPalaceAdFree()
{
    return _sim.Halls &gt;= PalaceSim.HallSlotCap
        &amp;&amp; _sim.RescueCount == 0
        &amp;&amp; _sim.DoubleCount == 0;
}
/// &lt;summary&gt;Longest run of consecutive days that each carried at
/// least one wheel spin (the "attended court" beat). Day index
/// derives from the sim event clock (DayIndexOf), never wall time.&lt;/summary&gt;
```

```
public int LongestCourtStreak()
{
    var days = new SortedSet&lt;int&gt;();
    var events = _sim.Events;
    for (int i = 0; i &lt; events.Count; i++)
    {
        var e = events[i];
        if (e.Name == "spin") days.Add(PalaceSim.DayIndexOf(e.Time));
    }
    int best = 0, run = 0, prev = int.MinValue;
    foreach (int d in days)
    {
        run = (d == prev + 1) ? run + 1 : 1;
        if (run &gt; best) best = run;
        prev = d;
    }
    return best;
}
/// &lt;summary&gt;Wall progress readout (slice-b host renders it): current
/// value clamped to the medal target, straight off the local state.&lt;/summary&gt;
public int ProgressOf(string id)
{
    switch (id)
    {
        case FirstHaul: return Math.Min(1, CountHauls());
        case FirstRank: return Math.Min(1, _sim.CountEvents("rank_up"));
        case ZeroAdClear: return Math.Min(PalaceSim.HallSlotCap, _sim.Halls);
        case AlbumHalf: return Math.Min(G18Catalogs.AlbumItemCount / 2, AlbumFlow.RowsLitOf(_sim.GalleryIds));
        case AlbumFull: return Math.Min(G18Catalogs.AlbumItemCount, AlbumFlow.RowsLitOf(_sim.GalleryIds));
        case CourtWeek: return Math.Min(CourtStreakDays, LongestCourtStreak());
        case RevengeTen: return Math.Min(RevengeTenCount, _sim.RevengeCount);
        case VeteranHundred: return Math.Min(VeteranRaidCount, _sim.RaidCount);
    }
    return 0;
}
/// &lt;summary&gt;Medal target (0/1 for the moment medals, the frozen
/// thresholds for the counters).&lt;/summary&gt;
public int TargetOf(string id)
{
    switch (id)
    {
        case FirstHaul:
        case FirstRank: return 1;
        case ZeroAdClear: return PalaceSim.HallSlotCap;
        case AlbumHalf: return G18Catalogs.AlbumItemCount / 2;
        case AlbumFull: return G18Catalogs.AlbumItemCount;
        case CourtWeek: return CourtStreakDays;
        case RevengeTen: return RevengeTenCount;
        case VeteranHundred: return VeteranRaidCount;
    }
```

```
            return 0;
        }
        /// &lt;summary&gt;i18n title key of a medal (red line 4: player-facing text
        /// stays externalized; the key shape is the medal id itself).&lt;/summary&gt;
        public static string TitleKeyOf(string id)
        {
            return string.IsNullOrEmpty(id) ? "" : id;
        }
        int CountHauls()
        {
            int n = 0;
            var events = _sim.Events;
            for (int i = 0; i &lt; events.Count; i++)
            {
                var e = events[i];
                if (e.Name == "raid_report" &amp;&amp; e.Detail != null &amp;&amp;
                    e.Detail.StartsWith("verdict=haul", StringComparison.Ordinal)) n++;
            }
            return n;
        }
    }
}
// AchievementsWallViewHost.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
using Logger = Sy.Core.Logger;
namespace Sy.Palace
{
    /// &lt;summary&gt;
    /// Scene wiring for the eight-achievement wall (task S2-T002 slice b,
    /// HallViewHost paradigm): installs on the boot scene next to the ring
    /// hosts and SHARES the wheel's PalaceSim session (one purse, one clock -
    /// the wheel owns AdvanceClock; a standalone boot owns it here), keeps a
    /// small always-on corner tab as the wall entry, and renders the eight
    /// medals per the slice-a AchievementFlow view-model verbatim (zero sim
    /// changes, zero network faces - red line 12: local progress only, the
    /// presenter never invents numbers). Every unlock lands as the registered
    /// telemetry beat (PalaceEventIds.AchievementUnlock, one beat per medal,
    /// monotone - a medal never un-locks so it never re-books) on the
    /// session's single funnel sink (the raid host owns it when the full
    /// ring is up; a standalone boot creates its own, resolved lazily
    /// because Start order across hosts is undefined). The wall face is the
    /// programmatic fallback the taskbook mandates: medal title (i18n
    /// externalized, red line 4), live progress "current/target" straight
    /// off the flow readouts, the bestowed/pending state chip, and the
    /// unlocked-row gilded tint. UGUI is built entirely from code (G17
```

```
/// law, zero scene assets).
/// &lt;/summary&gt;
public sealed class AchievementsWallViewHost : MonoBehaviour
{
    public const string SceneName = "S00_Boot";
    public const int WallRows = AchievementFlow.MedalCount;
    // i18n keys the face reads (red line 4: copy stays externalized)
    public const string TabKey = "g18.ach.tab";
    public const string TitleKey = "g18.ach.title";
    public const string CountKey = "g18.ach.count";
    public const string CloseKey = "g18.ach.close";
    // ---- GDD-frozen palette (WheelViewHost family) ----
    static readonly Color NightInk = G18Ui.Hex(0x1F2229);
    static readonly Color SilkWhite = G18Ui.Hex(0xF5EFDC);
    static readonly Color GuideYellow = G18Ui.Hex(0xFFD24A);
    static readonly Color WallRed = G18Ui.Hex(0x9F2B2B);
    static readonly Color GoldMain = G18Ui.Hex(0xD4AF37);
    static readonly Color ButtonAltBg = G18Ui.Hex(0x3A3F4A);
    static readonly Color RowDarkBg = G18Ui.Hex(0x2A2E38);
    const float RefW = 1080f, RefH = 1920f;
    const float RowStep = 140f, RowTop = 1250f;   // eight rows 1250..270, clear of the wall band and the close butt
    // ---- pure mapping faces (EditMode-testable, presenter never invents) ----
    /// &lt;summary&gt;Medal-row state chip key: bestowed or pending.&lt;/summary&gt;
    public static string RowStateKeyOf(bool unlocked)
    {
        return unlocked ? "g18.ach.row.unlocked" : "g18.ach.row.locked";
    }
    /// &lt;summary&gt;Progress readout "current/target", invariant format
    /// (culture-safe on every device locale).&lt;/summary&gt;
    public static string ProgressTextOf(int progress, int target)
    {
        return progress.ToString(System.Globalization.CultureInfo.InvariantCulture)
            + "/" + target.ToString(System.Globalization.CultureInfo.InvariantCulture);
    }
    /// &lt;summary&gt;Medal title color: gilded when bestowed, plain silk while
    /// pending (the wall reads at a glance).&lt;/summary&gt;
    public static Color RowNameColorOf(bool unlocked)
    {
        return unlocked ? GoldMain : SilkWhite;
    }
    /// &lt;summary&gt;Row backdrop: a faint gilded tint on bestowed rows, the
    /// dark block on pending ones.&lt;/summary&gt;
    public static Color RowBgColorOf(bool unlocked)
    {
        return unlocked
            ? new Color(GoldMain.r, GoldMain.g, GoldMain.b, 0.18f)
            : RowDarkBg;
    }
    /// &lt;summary&gt;Canvas Y of wall row i (top to bottom, fixed ladder).&lt;/summary&gt;
    public static float RowYOf(int row)
```

```
{
    return RowTop - row * RowStep;
}
PalaceSim _sim;
AchievementFlow _flow;
PalaceAdFunnel _funnel;
bool _ownsSim;
readonly HashSet&lt;string&gt; _reported = new HashSet&lt;string&gt;();
RectTransform _canvasRect;
GameObject _panelRoot;
GameObject _tabRoot;
Text _count;
Text[] _names;
Text[] _progress;
Text[] _states;
Image[] _rowBg;
// ---- test/inspector faces ----
public PalaceSim Sim { get { return _sim; } }
public AchievementFlow Flow { get { return _flow; } }
public bool TabVisible { get { return _tabRoot != null &amp;&amp; _tabRoot.activeSelf; } }
public bool PanelOpen { get { return _panelRoot != null &amp;&amp; _panelRoot.activeSelf; } }
public string CountText { get { return _count == null ? "" : _count.text; } }
public string RowNameText(int row)
{
    return _names == null || row &lt; 0 || row &gt;= _names.Length || _names[row] == null ? "" : _names[row].tex
}
public string RowProgressText(int row)
{
    return _progress == null || row &lt; 0 || row &gt;= _progress.Length || _progress[row] == null ? "" : _progr
}
public string RowStateText(int row)
{
    return _states == null || row &lt; 0 || row &gt;= _states.Length || _states[row] == null ? "" : _states[row]
}
/// &lt;summary&gt;The session's single telemetry sink (the raid host owns
/// the funnel when the full ring is up; resolved lazily because the
/// Start order across hosts is undefined - the first beat lands long
/// after every host has started).&lt;/summary&gt;
public PalaceAdFunnel Funnel
{
    get
    {
        if (_funnel == null)
        {
            var raid = FindObjectOfType&lt;RaidViewHost&gt;();
            _funnel = raid != null &amp;&amp; raid.Funnel != null ? raid.Funnel : new PalaceAdFunnel(_sim);
        }
        return _funnel;
    }
}
```

```
// ---------------- scene wiring (HallViewHost paradigm) ----------------
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
static void AutoInstall()
{
    SceneManager.sceneLoaded -= OnSceneLoaded;
    SceneManager.sceneLoaded += OnSceneLoaded;
}
static void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    if (scene.name != SceneName) return;
    if (FindObjectOfType&lt;AchievementsWallViewHost&gt;() != null) return;
    var go = new GameObject("AchievementsWallViewHost");
    go.AddComponent&lt;AchievementsWallViewHost&gt;();
}
void Awake()
{
    G18Text.Load();
    BuildUi();
    Logger.Info("G18.View", "achievements wall installed (S2-T002 slice b scene wiring online)");
}
void Start()
{
    // ring coexistence law, bound in Start so every host of the scene
    // has Awakened: the wheel host owns the shared session (one purse,
    // one clock); a standalone boot owns it here.
    var wheel = FindObjectOfType&lt;WheelViewHost&gt;();
    if (wheel != null &amp;&amp; wheel.Sim != null)
    {
        _sim = wheel.Sim;
        _ownsSim = false;
    }
    else
    {
        _sim = new PalaceSim(WheelViewHost.SimSeed);
        _sim.Open();
        _ownsSim = true;
    }
    _flow = new AchievementFlow(_sim);
}
void Update()
{
    if (_flow == null) return;
    if (_ownsSim) _sim.AdvanceClock(Time.deltaTime);
    _flow.Refresh();        // idempotent + monotone (the wall only grows)
    ReportNewUnlocks();     // one telemetry beat per medal, exactly once
    if (PanelOpen) RefreshRows();
}
// ---------------- test hooks = the one true op routers ----------------
/// &lt;summary&gt;Open the wall panel (the corner tab's op).&lt;/summary&gt;
public bool TestOpenWall()
```

```
{
    if (_panelRoot == null) return false;
    _panelRoot.SetActive(true);
    RefreshRows();
    return true;
}
public bool TestCloseWall()
{
    if (_panelRoot == null) return false;
    _panelRoot.SetActive(false);
    return true;
}
// ---------------- unlock beats + face refresh (read-only) ----------------
void ReportNewUnlocks()
{
    var all = AchievementFlow.All;
    for (int i = 0; i &lt; all.Length; i++)
    {
        string id = all[i];
        if (_reported.Contains(id) || !_flow.IsUnlocked(id)) continue;
        _reported.Add(id);          // book before the beat: monotone ids never re-report
        Funnel.ReportAchievementUnlock(id);
    }
}
void RefreshRows()
{
    if (_flow == null || _names == null) return;
    var all = AchievementFlow.All;
    for (int i = 0; i &lt; _names.Length &amp;&amp; i &lt; all.Length; i++)
    {
        string id = all[i];
        bool unlocked = _flow.IsUnlocked(id);
        if (_names[i] != null)
        {
            _names[i].text = G18Text.Get(AchievementFlow.TitleKeyOf(id));
            _names[i].color = RowNameColorOf(unlocked);
        }
        if (_progress[i] != null)
            _progress[i].text = ProgressTextOf(_flow.ProgressOf(id), _flow.TargetOf(id));
        if (_states[i] != null)
            _states[i].text = G18Text.Get(RowStateKeyOf(unlocked));
        if (_rowBg[i] != null)
            _rowBg[i].color = RowBgColorOf(unlocked);
    }
    if (_count != null)
        _count.text = string.Format(G18Text.Get(CountKey), _flow.UnlockedCount, AchievementFlow.MedalCount);
}
// ---------------- UGUI construction (all from code, zero scene assets) ----------------
void BuildUi()
{
```

```
var root = new GameObject("PalaceAchievementsCanvas", typeof(Canvas));
var canvas = root.GetComponent&lt;Canvas&gt;();
canvas.renderMode = RenderMode.ScreenSpaceOverlay;
canvas.sortingOrder = 13;   // the wall rides above the album panel, under the decree layer
var scaler = root.AddComponent&lt;CanvasScaler&gt;();
scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
scaler.referenceResolution = new Vector2(RefW, RefH);
scaler.matchWidthOrHeight = 0.5f;
root.AddComponent&lt;GraphicRaycaster&gt;();
_canvasRect = (RectTransform)root.transform;
// corner tab = the wall entry (always on; sits above the album tab)
_tabRoot = new GameObject("ach_tab", typeof(RectTransform), typeof(Image), typeof(Button));
var tabRect = (RectTransform)_tabRoot.transform;
tabRect.SetParent(_canvasRect, false);
tabRect.sizeDelta = new Vector2(250f, 110f);
tabRect.anchoredPosition = ToLocal(new Vector2(180f, 880f));
_tabRoot.GetComponent&lt;Image&gt;().color = WallRed;
var tab = _tabRoot.GetComponent&lt;Button&gt;();
tab.onClick.AddListener(delegate { TestOpenWall(); });
var tabLabel = G18Ui.Label(tabRect, G18Text.Get(TabKey), 38, SilkWhite, TextAnchor.MiddleCenter, true);
tabLabel.rectTransform.sizeDelta = new Vector2(230f, 96f);
tabLabel.rectTransform.anchoredPosition = Vector2.zero;
// full-screen wall panel (hidden until the tab opens it)
_panelRoot = new GameObject("ach_panel", typeof(RectTransform), typeof(Image));
var panelRect = (RectTransform)_panelRoot.transform;
panelRect.SetParent(_canvasRect, false);
panelRect.sizeDelta = new Vector2(RefW, RefH);
panelRect.anchoredPosition = Vector2.zero;
var panelBg = _panelRoot.GetComponent&lt;Image&gt;();
panelBg.color = NightInk;
panelBg.raycastTarget = true;    // panel swallows taps (modal ring face)
var wall = new GameObject("wall_band", typeof(RectTransform), typeof(Image));
var wallRect = (RectTransform)wall.transform;
wallRect.SetParent(panelRect, false);
wallRect.sizeDelta = new Vector2(RefW, 260f);
wallRect.anchoredPosition = ToLocal(new Vector2(RefW * 0.5f, 1435f));
wall.GetComponent&lt;Image&gt;().color = WallRed;
_panelRoot.SetActive(false);
var title = G18Ui.Label(panelRect, G18Text.Get(TitleKey), 60, GuideYellow, TextAnchor.UpperCenter, true);
title.rectTransform.sizeDelta = new Vector2(1000f, 110f);
title.rectTransform.anchoredPosition = ToLocal(new Vector2(RefW * 0.5f, 1750f));
_count = G18Ui.Label(panelRect, "", 44, SilkWhite, TextAnchor.MiddleCenter, true);
_count.rectTransform.sizeDelta = new Vector2(1000f, 80f);
_count.rectTransform.anchoredPosition = ToLocal(new Vector2(RefW * 0.5f, 1615f));
// eight medal rows (fixed ladder top to bottom, programmatic fallback)
_names = new Text[WallRows];
_progress = new Text[WallRows];
_states = new Text[WallRows];
_rowBg = new Image[WallRows];
for (int i = 0; i &lt; WallRows; i++)
```

```
            {
                float y = RowYOf(i);
                var bgGo = new GameObject("row_" + i, typeof(RectTransform), typeof(Image));
                var bgRect = (RectTransform)bgGo.transform;
                bgRect.SetParent(panelRect, false);
                bgRect.sizeDelta = new Vector2(980f, 120f);
                bgRect.anchoredPosition = ToLocal(new Vector2(RefW * 0.5f, y - 10f));
                var bg = bgGo.GetComponent&lt;Image&gt;();
                bg.color = RowDarkBg;
                bg.raycastTarget = false;
                _rowBg[i] = bg;
                _names[i] = G18Ui.Label(panelRect, "", 42, SilkWhite, TextAnchor.MiddleLeft, true);
                _names[i].rectTransform.sizeDelta = new Vector2(560f, 64f);
                _names[i].rectTransform.anchoredPosition = ToLocal(new Vector2(490f, y + 18f));
                _progress[i] = G18Ui.Label(panelRect, "", 34, GuideYellow, TextAnchor.MiddleLeft, false);
                _progress[i].rectTransform.sizeDelta = new Vector2(560f, 50f);
                _progress[i].rectTransform.anchoredPosition = ToLocal(new Vector2(490f, y - 45f));
                _states[i] = G18Ui.Label(panelRect, "", 36, SilkWhite, TextAnchor.MiddleCenter, true);
                _states[i].rectTransform.sizeDelta = new Vector2(280f, 64f);
                _states[i].rectTransform.anchoredPosition = ToLocal(new Vector2(910f, y + 18f));
            }
            // close button (back to the court ring)
            var close = TextButton(panelRect, "close", G18Text.Get(CloseKey),
                ToLocal(new Vector2(RefW * 0.5f, 90f)), new Vector2(360f, 110f), ButtonAltBg);
            close.onClick.AddListener(delegate { TestCloseWall(); });
        }
        static Vector2 ToLocal(Vector2 canvasPos)
        {
            return canvasPos - new Vector2(RefW, RefH) * 0.5f;
        }
        static Button TextButton(RectTransform parent, string name, string label, Vector2 localPos, Vector2 size, Color
        {
            var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
            var rect = (RectTransform)go.transform;
            rect.SetParent(parent, false);
            rect.sizeDelta = size;
            rect.anchoredPosition = localPos;
            go.GetComponent&lt;Image&gt;().color = bg;
            var button = go.GetComponent&lt;Button&gt;();
            var text = G18Ui.Label(rect, label, 38, NightInk, TextAnchor.MiddleCenter, true);
            text.rectTransform.sizeDelta = new Vector2(size.x - 20f, size.y - 16f);
            text.rectTransform.anchoredPosition = Vector2.zero;
            return button;
        }
    }
}
// AlbumFlow.cs
using System;
using System.Collections.Generic;
namespace Sy.Palace
```

```
{
    /// &lt;summary&gt;
    /// Pure view-model for the curio album face (task S1-T005 slice a,
    /// RaidFlow paradigm, zero sim changes). Faces per the taskbook spec
    /// block: the collected count off the sim's distinct GalleryIds ledger,
    /// the distance-to-complete progress anchor (the "only X to a full
    /// album" readout = AlbumItemCount minus collected, the locked 24-item
    /// face in G18Catalogs), the latest gallery_add entry parsed off the
    /// event stream, and the four-tier attribution of a collected curio (the
    /// trailing tier digit of the sim curio id maps onto the catalog tier
    /// names, table freeze order 1..4 = bronze / silver / gold / jade).
    /// Id-space note (honest bookkeeping, C986 precedent): the sim curio ids
    /// (CR-target-tier form) and the album table row ids (B01.. / S.. / G..
    /// / J..) are two different spaces - RowOf answers the catalog law
    /// (unknown = null) for sim curios; the wall-grid row consumption is the
    /// slice-b ViewHost seat. The id-space reconciliation landed with the
    /// S3-T003 slice a row-lit law: within a tier the Nth distinct collected
    /// curio lights catalog row N (capped at AlbumPerTier), exposed as
    /// LitOfTier / RowsLitOf - the face the anchor and the album
    /// achievements consume.
    /// &lt;/summary&gt;
    public class AlbumFlow
    {
        readonly PalaceSim _sim;
        public AlbumFlow(PalaceSim sim)
        {
            if (sim == null) throw new ArgumentNullException("sim");
            _sim = sim;
        }
        public PalaceSim Sim { get { return _sim; } }
        /// &lt;summary&gt;Distinct curios collected (sim ledger parity).&lt;/summary&gt;
        public int Collected { get { return _sim.GalleryIds.Count; } }
        /// &lt;summary&gt;Total album face (G18Catalogs locked 24).&lt;/summary&gt;
        public int TotalItems { get { return G18Catalogs.AlbumItemCount; } }
        /// &lt;summary&gt;Progress anchor: distance to a complete album, read off
        /// the row-lit face (task S3-T003 slice a: lit wall rows, not the raw
        /// distinct-id count - 24 distinct ids spread unevenly across tiers
        /// light fewer than 24 wall rows, so the raw count over-promises).&lt;/summary&gt;
        public int RemainingToComplete
        {
            get
            {
                int r = G18Catalogs.AlbumItemCount - RowsLit;
                return r &lt; 0 ? 0 : r;
            }
        }
        /// &lt;summary&gt;Lit wall rows across the four catalog tiers (0..
        /// AlbumItemCount): per tier the Nth distinct curio lights catalog
        /// row N, capped at the six rows the tier owns - duplicate finds and
        /// over-tier variants never light a seventh row (S3-T003 slice a
```

```
/// id-space reconciliation law, the bookkeeping seat the slice-a
/// header note reserved).&lt;/summary&gt;
public int RowsLit { get { return RowsLitOf(_sim.GalleryIds); } }
/// &lt;summary&gt;Lit wall rows of one tier (pure static,
/// EditMode-testable): min(distinct collected ids of the tier, rows
/// the tier owns in the catalog). Unknown-tier ids light nothing.&lt;/summary&gt;
public static int LitOfTier(ICollection&lt;string&gt; galleryIds, string tier)
{
    if (galleryIds == null || galleryIds.Count == 0) return 0;
    int distinct = 0;
    foreach (var id in galleryIds)
        if (TierOfCurio(id) == tier) distinct++;
    int cap = G18Catalogs.AlbumPerTier;
    return distinct &lt; cap ? distinct : cap;
}
/// &lt;summary&gt;Lit wall rows across the four catalog tiers - the one
/// honest album face the anchor readout and the album achievements
/// both consume (wall/achievement parity, S3-T003 slice a).&lt;/summary&gt;
public static int RowsLitOf(ICollection&lt;string&gt; galleryIds)
{
    if (galleryIds == null || galleryIds.Count == 0) return 0;
    return LitOfTier(galleryIds, G18Catalogs.TierBronze)
        + LitOfTier(galleryIds, G18Catalogs.TierSilver)
        + LitOfTier(galleryIds, G18Catalogs.TierGold)
        + LitOfTier(galleryIds, G18Catalogs.TierJade);
}
/// &lt;summary&gt;Latest curio added (event-stream parsed; empty before the first haul).&lt;/summary&gt;
public string LastCurio
{
    get
    {
        var e = LastEvent("gallery_add");
        return e == null ? "" : FieldOf(e.Detail, "curio");
    }
}
/// &lt;summary&gt;gallery_add entries observed so far (event-stream face).&lt;/summary&gt;
public int GalleryAddObserved { get { return _sim.CountEvents("gallery_add"); } }
/// &lt;summary&gt;Four-tier attribution: the trailing tier digit of a sim curio
/// id maps onto the catalog tier names (1..4 = bronze / silver / gold /
/// jade, the table freeze order). The flow never invents tiers - unknown
/// digits read as empty.&lt;/summary&gt;
public static string TierOfCurio(string curio)
{
    if (string.IsNullOrEmpty(curio) || !curio.StartsWith("CR-")) return "";
    int ix = curio.LastIndexOf('-');
    if (ix &lt; 0 || ix + 1 &gt;= curio.Length) return "";
    int d = curio[ix + 1] - '0';
    switch (d)
    {
        case 1: return G18Catalogs.TierBronze;
```

```
                case 2: return G18Catalogs.TierSilver;
                case 3: return G18Catalogs.TierGold;
                case 4: return G18Catalogs.TierJade;
                default: return "";
            }
        }
        /// &lt;summary&gt;Album table row for a row id (B01../S../G../J.. space) - the
        /// wall-grid lookup face. Unknown = null, catalog law.&lt;/summary&gt;
        public AlbumRow RowOf(string id)
        {
            return G18Catalogs.AlbumById(G18Catalogs.LoadAlbum(), id);
        }
        /// &lt;summary&gt;The nth row of one tier in table freeze order (0-based,
        /// task S3-T003 slice b: the named-row wall renders tier rows
        /// through this face so a lit row N always shows the same frozen
        /// catalog curio name - never an invented slot). Unknown tier or
        /// out-of-range index reads null; a null file reads null.&lt;/summary&gt;
        public static AlbumRow RowIndexOfTier(AlbumFile file, string tier, int n)
        {
            if (file == null || file.items == null || n &lt; 0) return null;
            int seen = 0;
            for (int i = 0; i &lt; file.items.Length; i++)
            {
                if (file.items[i].tier != tier) continue;
                if (seen == n) return file.items[i];
                seen++;
            }
            return null;
        }
        // ---- event-stream helpers (read-only, presenter never re-rolls) ----
        PalaceSim.SimEvent LastEvent(string name)
        {
            var events = _sim.Events;
            for (int i = events.Count - 1; i &gt;= 0; i--)
                if (events[i].Name == name) return events[i];
            return null;
        }
        static string FieldOf(string detail, string key)
        {
            if (string.IsNullOrEmpty(detail)) return "";
            string token = key + "=";
            int ix = detail.IndexOf(token, StringComparison.Ordinal);
            if (ix &lt; 0) return "";
            int start = ix + token.Length;
            int end = start;
            while (end &lt; detail.Length &amp;&amp; detail[end] != ' ') end++;
            return detail.Substring(start, end - start);
        }
    }
}
```

```
// AlbumViewHost.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
using Logger = Sy.Core.Logger;
namespace Sy.Palace
{
    /// &lt;summary&gt;
    /// Scene wiring for the curio album face (task S1-T005 slice b,
    /// RaidViewHost paradigm): installs on the boot scene next to the wheel,
    /// raid and revenge hosts and SHARES the PalaceSim session (one purse,
    /// one clock - the wheel owns AdvanceClock when present; a standalone
    /// boot owns it here). Faces per the taskbook spec block: the collected
    /// count + distance-to-complete progress anchor (the "only X to a full
    /// album" readout, locked 24-item face in G18Catalogs), the four-tier
    /// wall (bronze / silver / gold / jade rows rendered as procedural
    /// color-block fallback per the tier digit of the sim curio ids - the
    /// U020 legal placeholder face), and the curio lightbox (the
    /// first-haul collection beat: a gallery_add event pops the new curio
    /// with its tier attribution), plus the ArtRuntime fallback chain
    /// (stem = scene_palace_album). Zero sim changes: every number reads
    /// from the AlbumFlow view-model (slice a) or the sim ledger/event
    /// stream.
    /// S3-T003 slice b (named-row wall): every tier band now renders its six
    /// frozen catalog rows as named chips - the nth chip of a tier shows the
    /// nth table-freeze curio name (AlbumFlow.RowIndexOfTier) and lights
    /// through the slice-a row-lit law (the nth distinct collected curio of
    /// the tier lights row n, capped at six). Unlit chips keep the curio
    /// name visible at a dimmed silhouette face - the collection-goal face
    /// the parallel growth line needs; no invented slots, no invented names.
    /// &lt;/summary&gt;
    public sealed class AlbumViewHost : MonoBehaviour
    {
        public const string SceneName = "S00_Boot";
        public const string AlbumStem = "scene_palace_album";
        // ---- GDD-frozen palette (WheelViewHost family) ----
        static readonly Color NightInk = G18Ui.Hex(0x1F2229);
        static readonly Color SilkWhite = G18Ui.Hex(0xF5EFDC);
        static readonly Color GuideYellow = G18Ui.Hex(0xFFD24A);
        static readonly Color GoldMain = G18Ui.Hex(0xD4AF37);
        static readonly Color ButtonAltBg = G18Ui.Hex(0x3A3F4A);
        static readonly Color PanelBg = new Color(0.06f, 0.06f, 0.08f, 0.94f);
        static readonly Color FallbackBlock = G18Ui.Hex(0x555963);
        const float RefW = 1080f, RefH = 1920f;
        // ---- pure mapping faces (EditMode-testable, presenter never invents) ----
        /// &lt;summary&gt;Procedural wall color of a catalog tier (four-tier wall
        /// color-block fallback; unknown tiers read the neutral block).&lt;/summary&gt;
```

```
public static Color TierColorOf(string tier)
{
    if (tier == G18Catalogs.TierBronze) return G18Ui.Hex(0xB08D57);
    if (tier == G18Catalogs.TierSilver) return G18Ui.Hex(0xC7CCD4);
    if (tier == G18Catalogs.TierGold) return GuideYellow;
    if (tier == G18Catalogs.TierJade) return G18Ui.Hex(0x2E8B57);
    return FallbackBlock;
}
/// &lt;summary&gt;Tier display i18n key of a catalog tier name.&lt;/summary&gt;
public static string TierNameKeyOf(string tier)
{
    if (tier == G18Catalogs.TierBronze) return "g18.album.tier.bronze";
    if (tier == G18Catalogs.TierSilver) return "g18.album.tier.silver";
    if (tier == G18Catalogs.TierGold) return "g18.album.tier.gold";
    if (tier == G18Catalogs.TierJade) return "g18.album.tier.jade";
    return "";
}
/// &lt;summary&gt;Collected count of one tier off the sim gallery ledger:
/// the CR-&amp;lt;target&amp;gt;-&amp;lt;tier digit&amp;gt; ids group through the
/// slice-a AlbumFlow tier law (pure static, EditMode-testable).&lt;/summary&gt;
public static int CountOfTier(IEnumerable&lt;string&gt; galleryIds, string tier)
{
    if (galleryIds == null) return 0;
    int n = 0;
    foreach (var id in galleryIds)
        if (AlbumFlow.TierOfCurio(id) == tier) n++;
    return n;
}
/// &lt;summary&gt;Wall row lit law (slice-b wall face over the slice-a
/// statics): catalog row N of a tier is lit iff the tier holds more
/// than N distinct collected curios (the slice-a Nth-distinct law),
/// and never beyond the six rows the tier owns. Input face = the
/// sim's distinct gallery set (GalleryIds) - the face the wall and
/// the slice-a statics consume. Pure static, EditMode-testable; the
/// wall chips and this face read the same law so the visual and the
/// readout can never drift.&lt;/summary&gt;
public static bool RowIsLit(ICollection&lt;string&gt; galleryIds, string tier, int row)
{
    if (row &lt; 0 || row &gt;= G18Catalogs.AlbumPerTier) return false;
    return row &lt; AlbumFlow.LitOfTier(galleryIds, tier);
}
/// &lt;summary&gt;Unlit chip silhouette color of a tier (the dimmed
/// goal face): the tier wall color at a low alpha, so an unlit
/// named row still hints at its tier while staying clearly off.&lt;/summary&gt;
public static Color ChipDimColorOf(string tier)
{
    Color c = TierColorOf(tier);
    c.a = 0.28f;
    return c;
}
```

```
PalaceSim _sim;
AlbumFlow _flow;
bool _ownsSim;
RectTransform _canvasRect;
GameObject _panelRoot;
GameObject _tabRoot;
Text _collected;
Text _anchor;
Text[] _tierCounts;
Image[] _tierBlocks;
Image[] _chipBlocks;    // slice-b named-row wall: 4 tiers x 6 chips, flattened
Text[] _chipNames;
GameObject _lightboxRoot;
Text _lightboxCurio;
Text _lightboxTier;
Text _lightboxAnchor;
int _eventCursor;
// ---- test/inspector faces ----
public PalaceSim Sim { get { return _sim; } }
public AlbumFlow Flow { get { return _flow; } }
public bool PanelOpen { get { return _panelRoot != null &amp;&amp; _panelRoot.activeSelf; } }
public bool TabVisible { get { return _tabRoot != null &amp;&amp; _tabRoot.activeSelf; } }
public bool LightboxShown { get { return _lightboxRoot != null &amp;&amp; _lightboxRoot.activeSelf; } }
public string CollectedText { get { return _collected == null ? "" : _collected.text; } }
public string AnchorText { get { return _anchor == null ? "" : _anchor.text; } }
public string LightboxCurioText { get { return _lightboxCurio == null ? "" : _lightboxCurio.text; } }
public string TierCountText(int row) { return _tierCounts == null || row &lt; 0 || row &gt;= _tierCounts.Length
/// &lt;summary&gt;Named chip count of the wall (4 tiers x 6 rows, slice b).&lt;/summary&gt;
public int ChipCount { get { return _chipBlocks == null ? 0 : _chipBlocks.Length; } }
/// &lt;summary&gt;Chip name text of one wall seat (tierIx 0..3, row 0..5) -
/// the frozen catalog curio name through the table nameKey.&lt;/summary&gt;
public string TierChipText(int tierIx, int row)
{
    if (_chipNames == null || tierIx &lt; 0 || tierIx &gt;= 4) return "";
    int ix = tierIx * G18Catalogs.AlbumPerTier + row;
    return ix &lt; 0 || ix &gt;= _chipNames.Length || _chipNames[ix] == null ? "" : _chipNames[ix].text;
}
/// &lt;summary&gt;Chip lit state of one wall seat - recomputed off the
/// row-lit law every read, so the face can never drift from the
/// chips the wall renders.&lt;/summary&gt;
public bool TierChipLit(int tierIx, int row)
{
    if (_sim == null || tierIx &lt; 0 || tierIx &gt;= 4) return false;
    return RowIsLit(_sim.GalleryIds, TierByIx(tierIx), row);
}
static string TierByIx(int tierIx)
{
    switch (tierIx)
    {
        case 0: return G18Catalogs.TierBronze;
```

```
        case 1: return G18Catalogs.TierSilver;
        case 2: return G18Catalogs.TierGold;
        default: return G18Catalogs.TierJade;
    }
}
// ---------------- scene wiring (WheelViewHost paradigm) ----------------
[RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
static void AutoInstall()
{
    SceneManager.sceneLoaded -= OnSceneLoaded;
    SceneManager.sceneLoaded += OnSceneLoaded;
}
static void OnSceneLoaded(Scene scene, LoadSceneMode mode)
{
    if (scene.name != SceneName) return;
    if (FindObjectOfType&lt;AlbumViewHost&gt;() != null) return;
    var go = new GameObject("AlbumViewHost");
    go.AddComponent&lt;AlbumViewHost&gt;();
}
void Awake()
{
    G18Text.Load();
    BuildUi();
    Logger.Info("G18.View", "album view installed (S1-T005 slice b scene wiring online)");
}
void Start()
{
    // ring coexistence law, bound in Start so every host of the scene
    // has Awakened: the wheel host owns the shared session (one purse,
    // one clock); a standalone boot owns it here.
    var wheel = FindObjectOfType&lt;WheelViewHost&gt;();
    if (wheel != null &amp;&amp; wheel.Sim != null)
    {
        _sim = wheel.Sim;
        _ownsSim = false;
    }
    else
    {
        _sim = new PalaceSim(20260924);
        _sim.Open();
        _ownsSim = true;
    }
    _flow = new AlbumFlow(_sim);
    _eventCursor = _sim.Events.Count;   // shared session: history stays with its owner ring
}
void Update()
{
    if (_flow == null) return;
    if (_ownsSim) _sim.AdvanceClock(Time.deltaTime);
    DrainEvents();
```

```
    RefreshFaces();
}
void DrainEvents()
{
    var events = _sim.Events;
    while (_eventCursor &lt; events.Count)
    {
        var ev = events[_eventCursor++];
        if (ev.Name == "gallery_add") StageLightbox();
    }
}
// ---------------- test hooks = the one true op routers ----------------
/// &lt;summary&gt;Open the album panel (the corner tab's op).&lt;/summary&gt;
public bool TestOpenAlbum()
{
    if (_panelRoot == null) return false;
    _panelRoot.SetActive(true);
    RefreshFaces();
    return true;
}
public bool TestCloseAlbum()
{
    if (_panelRoot == null) return false;
    _panelRoot.SetActive(false);
    return true;
}
/// &lt;summary&gt;Dismiss the curio lightbox (the collection beat's close op).&lt;/summary&gt;
public bool TestCloseLightbox()
{
    if (_lightboxRoot == null) return false;
    _lightboxRoot.SetActive(false);
    return true;
}
// ---------------- face refresh (read-only, never re-rolls) ----------------
void RefreshFaces()
{
    // S3-T003 slice a: the panel and tier rows read the row-lit face
    // (lit wall rows per tier, capped at the six rows a tier owns) -
    // the raw distinct count could read 8/6 on a tier wall.
    if (_collected != null)
        _collected.text = string.Format(G18Text.Get("g18.album.collected"), _flow.RowsLit, _flow.TotalItems);
    if (_anchor != null)
        _anchor.text = string.Format(G18Text.Get("g18.album.anchor"), _flow.RemainingToComplete);
    if (_tierCounts != null &amp;&amp; _flow != null)
    {
        string[] tiers = { G18Catalogs.TierBronze, G18Catalogs.TierSilver, G18Catalogs.TierGold, G18Catalogs.Tie
        for (int i = 0; i &lt; _tierCounts.Length &amp;&amp; i &lt; tiers.Length; i++)
            _tierCounts[i].text = string.Format(G18Text.Get("g18.album.wall.count"),
                AlbumFlow.LitOfTier(_sim.GalleryIds, tiers[i]), G18Catalogs.AlbumPerTier);
        // slice b: chips light through the same row-lit law (single
```

```
            // law, zero drift between readout and wall)
            if (_chipBlocks != null)
            {
                for (int i = 0; i &lt; tiers.Length; i++)
                {
                    Color litColor = TierColorOf(tiers[i]);
                    Color dimColor = ChipDimColorOf(tiers[i]);
                    Color litName = NightInk;
                    Color dimName = SilkWhite;
                    dimName.a = 0.55f;
                    for (int r = 0; r &lt; G18Catalogs.AlbumPerTier; r++)
                    {
                        int ix = i * G18Catalogs.AlbumPerTier + r;
                        if (_chipBlocks[ix] == null) continue;
                        bool isLit = RowIsLit(_sim.GalleryIds, tiers[i], r);
                        _chipBlocks[ix].color = isLit ? litColor : dimColor;
                        if (_chipNames[ix] != null)
                        {
                            _chipNames[ix].color = isLit ? litName : dimName;
                            _chipNames[ix].fontStyle = isLit ? FontStyle.Bold : FontStyle.Normal;
                        }
                    }
                }
            }
        }
        void StageLightbox()
        {
            if (_lightboxRoot == null) return;
            string curio = _flow.LastCurio;
            string tier = AlbumFlow.TierOfCurio(curio);
            string tierKey = TierNameKeyOf(tier);
            _lightboxCurio.text = curio;
            _lightboxTier.text = tierKey.Length &gt; 0 &amp;&amp; G18Text.Has(tierKey) ? G18Text.Get(tierKey) : tier;
            _lightboxAnchor.text = string.Format(G18Text.Get("g18.album.anchor"), _flow.RemainingToComplete);
            _lightboxRoot.SetActive(true);
        }
        // ---------------- UGUI construction (all from code, zero scene assets) ----------------
        void BuildUi()
        {
            var root = new GameObject("PalaceAlbumCanvas", typeof(Canvas));
            var canvas = root.GetComponent&lt;Canvas&gt;();
            canvas.renderMode = RenderMode.ScreenSpaceOverlay;
            canvas.sortingOrder = 12;   // album panel rides above the wheel/raid/revenge canvases
            var scaler = root.AddComponent&lt;CanvasScaler&gt;();
            scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
            scaler.referenceResolution = new Vector2(RefW, RefH);
            scaler.matchWidthOrHeight = 0.5f;
            root.AddComponent&lt;GraphicRaycaster&gt;();
            _canvasRect = (RectTransform)root.transform;
```

```
// corner tab = the ring entry (stacked above the revenge tab)
_tabRoot = new GameObject("album_tab", typeof(RectTransform), typeof(Image), typeof(Button));
var tabRect = (RectTransform)_tabRoot.transform;
tabRect.SetParent(_canvasRect, false);
tabRect.sizeDelta = new Vector2(250f, 110f);
tabRect.anchoredPosition = ToLocal(new Vector2(180f, 560f));
_tabRoot.GetComponent&lt;Image&gt;().color = GoldMain;
var tab = _tabRoot.GetComponent&lt;Button&gt;();
tab.onClick.AddListener(delegate { TestOpenAlbum(); });
var tabLabel = G18Ui.Label(tabRect, G18Text.Get("g18.album.tab"), 38, NightInk, TextAnchor.MiddleCenter, tru
tabLabel.rectTransform.sizeDelta = new Vector2(230f, 96f);
tabLabel.rectTransform.anchoredPosition = Vector2.zero;
// full-screen album panel (hidden until the tab opens it)
_panelRoot = new GameObject("album_panel", typeof(RectTransform), typeof(Image));
var panelRect = (RectTransform)_panelRoot.transform;
panelRect.SetParent(_canvasRect, false);
panelRect.sizeDelta = new Vector2(RefW, RefH);
panelRect.anchoredPosition = Vector2.zero;
var panelBg = _panelRoot.GetComponent&lt;Image&gt;();
panelBg.color = NightInk;
panelBg.raycastTarget = true;    // panel swallows taps (modal ring face)
var panelArt = ArtRuntime.Get(AlbumStem);
if (panelArt != null)
{
    panelBg.sprite = panelArt;
    panelBg.color = Color.white;
}
_panelRoot.SetActive(false);
var title = G18Ui.Label(panelRect, G18Text.Get("g18.album.title"), 60, GuideYellow, TextAnchor.UpperCenter,
title.rectTransform.sizeDelta = new Vector2(1000f, 110f);
title.rectTransform.anchoredPosition = ToLocal(new Vector2(RefW * 0.5f, 1750f));
_collected = G18Ui.Label(panelRect, "", 48, SilkWhite, TextAnchor.MiddleCenter, true);
_collected.rectTransform.sizeDelta = new Vector2(1000f, 90f);
_collected.rectTransform.anchoredPosition = ToLocal(new Vector2(RefW * 0.5f, 1600f));
_anchor = G18Ui.Label(panelRect, "", 42, GuideYellow, TextAnchor.MiddleCenter, false);
_anchor.rectTransform.sizeDelta = new Vector2(1000f, 80f);
_anchor.rectTransform.anchoredPosition = ToLocal(new Vector2(RefW * 0.5f, 1500f));
// four-tier named-row wall (slice b: header band + six frozen
// catalog chips per tier, procedural color-block fallback, U020 law)
string[] tiers = { G18Catalogs.TierBronze, G18Catalogs.TierSilver, G18Catalogs.TierGold, G18Catalogs.TierJad
var albumFile = G18Catalogs.LoadAlbum();
_tierCounts = new Text[tiers.Length];
_tierBlocks = new Image[tiers.Length];
_chipBlocks = new Image[tiers.Length * G18Catalogs.AlbumPerTier];
_chipNames = new Text[tiers.Length * G18Catalogs.AlbumPerTier];
for (int i = 0; i &lt; tiers.Length; i++)
{
    float top = 1430f - i * 295f;
    // tier header band (name + lit count, the slice-a contract face)
    var band = new GameObject("tier_band_" + tiers[i], typeof(RectTransform));
```

```
        var bandRect = (RectTransform)band.transform;
        bandRect.SetParent(panelRect, false);
        bandRect.sizeDelta = new Vector2(940f, 80f);
        bandRect.anchoredPosition = ToLocal(new Vector2(RefW * 0.5f, top - 40f));
        var block = band.AddComponent&lt;Image&gt;();
        block.color = TierColorOf(tiers[i]);
        _tierBlocks[i] = block;
        string tierKey = TierNameKeyOf(tiers[i]);
        var name = G18Ui.Label(bandRect, G18Text.Get(tierKey), 36, NightInk, TextAnchor.MiddleLeft, true);
        name.rectTransform.sizeDelta = new Vector2(400f, 64f);
        name.rectTransform.anchoredPosition = new Vector2(-250f, 0f);
        _tierCounts[i] = G18Ui.Label(bandRect, "", 36, NightInk, TextAnchor.MiddleRight, true);
        _tierCounts[i].rectTransform.sizeDelta = new Vector2(300f, 64f);
        _tierCounts[i].rectTransform.anchoredPosition = new Vector2(300f, 0f);
        // six named chips (2 rows x 3 cols): the nth chip shows the nth
        // frozen catalog curio of the tier and lights through the
        // slice-a row-lit law; unlit chips keep the name visible at a
        // dimmed silhouette face (the collection-goal face)
        for (int r = 0; r &lt; G18Catalogs.AlbumPerTier; r++)
        {
            int ix = i * G18Catalogs.AlbumPerTier + r;
            int col = r % 3, rowIx = r / 3;
            var chip = new GameObject("chip_" + tiers[i] + "_" + r, typeof(RectTransform));
            var chipRect = (RectTransform)chip.transform;
            chipRect.SetParent(panelRect, false);
            chipRect.sizeDelta = new Vector2(300f, 92f);
            chipRect.anchoredPosition = ToLocal(new Vector2(
                RefW * 0.5f - 310f + col * 310f, top - 135f - rowIx * 100f));
            _chipBlocks[ix] = chip.AddComponent&lt;Image&gt;();
            var catalogRow = AlbumFlow.RowIndexOfTier(albumFile, tiers[i], r);
            string chipName = catalogRow == null ? ""
                : (G18Text.Has(catalogRow.nameKey) ? G18Text.Get(catalogRow.nameKey) : catalogRow.id);
            var chipLabel = G18Ui.Label(chipRect, chipName, 26, SilkWhite, TextAnchor.MiddleCenter, false);
            chipLabel.rectTransform.sizeDelta = new Vector2(280f, 76f);
            chipLabel.rectTransform.anchoredPosition = Vector2.zero;
            _chipNames[ix] = chipLabel;
        }
    }
    // curio lightbox (first-haul collection beat)
    _lightboxRoot = new GameObject("curio_lightbox", typeof(RectTransform));
    var lightboxRect = (RectTransform)_lightboxRoot.transform;
    lightboxRect.SetParent(panelRect, false);
    lightboxRect.sizeDelta = new Vector2(940f, 460f);
    lightboxRect.anchoredPosition = ToLocal(new Vector2(RefW * 0.5f, 700f));
    var lightboxBg = _lightboxRoot.AddComponent&lt;Image&gt;();
    lightboxBg.color = PanelBg;
    lightboxBg.raycastTarget = false;
    var lightboxTitle = G18Ui.Label(lightboxRect, G18Text.Get("g18.album.lightbox.title"), 46, GuideYellow, Text
    lightboxTitle.rectTransform.sizeDelta = new Vector2(880f, 90f);
    lightboxTitle.rectTransform.anchoredPosition = new Vector2(0f, 150f);
```

```
            _lightboxCurio = G18Ui.Label(lightboxRect, "", 54, SilkWhite, TextAnchor.MiddleCenter, true);
            _lightboxCurio.rectTransform.sizeDelta = new Vector2(880f, 100f);
            _lightboxCurio.rectTransform.anchoredPosition = new Vector2(0f, 40f);
            _lightboxTier = G18Ui.Label(lightboxRect, "", 40, SilkWhite, TextAnchor.MiddleCenter, false);
            _lightboxTier.rectTransform.sizeDelta = new Vector2(880f, 80f);
            _lightboxTier.rectTransform.anchoredPosition = new Vector2(0f, -50f);
            _lightboxAnchor = G18Ui.Label(lightboxRect, "", 36, GuideYellow, TextAnchor.MiddleCenter, false);
            _lightboxAnchor.rectTransform.sizeDelta = new Vector2(880f, 70f);
            _lightboxAnchor.rectTransform.anchoredPosition = new Vector2(0f, -130f);
            _lightboxRoot.SetActive(false);
            // close button (back to the court)
            var close = TextButton(panelRect, "close", G18Text.Get("g18.album.close"),
                ToLocal(new Vector2(RefW * 0.5f, 120f)), new Vector2(360f, 110f), ButtonAltBg);
            close.onClick.AddListener(delegate { TestCloseAlbum(); });
        }
        static Vector2 ToLocal(Vector2 canvasPos)
        {
            return canvasPos - new Vector2(RefW, RefH) * 0.5f;
        }
        static Button TextButton(RectTransform parent, string name, string label, Vector2 localPos, Vector2 size, Color
        {
            var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
            var rect = (RectTransform)go.transform;
            rect.SetParent(parent, false);
            rect.sizeDelta = size;
            rect.anchoredPosition = localPos;
            go.GetComponent&lt;Image&gt;().color = bg;
            var button = go.GetComponent&lt;Button&gt;();
            var text = G18Ui.Label(rect, label, 38, NightInk, TextAnchor.MiddleCenter, true);
            text.rectTransform.sizeDelta = new Vector2(size.x - 20f, size.y - 16f);
            text.rectTransform.anchoredPosition = Vector2.zero;
            return button;
        }
    }
}
// G18Catalogs.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.Palace
{
    [Serializable]
    public sealed class WheelSegment
    {
        public string id;        // coins / timber / event_card / raid_ticket
        public string outcome;   // mirrors PalaceSim.WheelOutcome
        public int weight;       // percent, the four faces sum to 100
        public int payoutMin;    // frozen E4 band (numbers pack section 1)
        public int payoutMax;
```

```
    public string nameKey;   // i18n key (red line 4)
}
[Serializable]
public sealed class WheelFile
{
    public WheelSegment[] segments;
}
[Serializable]
public sealed class RankRow
{
    public string id;           // rank_1..rank_5
    public string nameKey;      // rank display name (locked C1017, U129 policy scan zero hits)
    public string buildingTier; // none / A03..A06 (four hall-building tiers)
    public int hallsPerRank;    // E4 graybox parity: every 3 halls promote one rank
}
[Serializable]
public sealed class RanksFile
{
    public string curveStatus;   // locked_s1_c1017 since the T003 slice-c numeric lock (numbers pack section 2 v1.0
    public int hallSlotCap;      // full-palace slot cap (12; mirrors PalaceSim.HallSlotCap)
    public int[] hallCoinCurve;  // per-slot build cost ladder, one entry per slot (parity with 8 + slot/5)
    public RankRow[] ranks;
}
[Serializable]
public sealed class RosterRow
{
    public string id;       // r01..r08
    public string token;    // English fiction token (sim-facing id, B6 fiction gate)
    public string nameKey;
}
[Serializable]
public sealed class RosterFile
{
    public RosterRow[] seats;
}
[Serializable]
public sealed class AlbumRow
{
    public string id;       // B01..B06 / S.. / G.. / J..
    public string tier;     // bronze / silver / gold / jade
    public string stem;     // art stem for the later art batch
    public string nameKey;
}
[Serializable]
public sealed class AlbumFile
{
    public AlbumRow[] items;
}
[Serializable]
public sealed class EdictRow
```

```
{
    public string id;        // e01..e10
    public string polarity;  // positive / absurd (numbers pack section 1 comedy dual state)
    public string textKey;
    public int bonusMin;     // frozen E4 band 4..8 (PalaceSim event-card leg)
    public int bonusMax;
}
[Serializable]
public sealed class EdictsFile
{
    public EdictRow[] events;
}
/// &lt;summary&gt;
/// Five frozen data tables for task S0-T003 (GDD section 8 data face):
/// wheel segments, rank ladder, preset roster, curio album and edict
/// events, plus their externalized i18n keys. Load returns null on a
/// missing or unparseable file and never throws; unknown id lookups
/// return null and are never an error (G12GameCatalog paradigm).
/// Validate() methods return the problem list; empty = well-formed.
/// All parity anchors trace to PalaceSim E4 values and the numbers
/// pack so the tables cannot drift from the frozen graybox face
/// before S1 tuning lands.
/// &lt;/summary&gt;
public static class G18Catalogs
{
    public const string WheelCoins = "coins";
    public const string WheelTimber = "timber";
    public const string WheelEventCard = "event_card";
    public const string WheelRaidTicket = "raid_ticket";
    public const string TierBronze = "bronze";
    public const string TierSilver = "silver";
    public const string TierGold = "gold";
    public const string TierJade = "jade";
    public const string PolarityPositive = "positive";
    public const string PolarityAbsurd = "absurd";
    public const int RankCount = 5;
    public const int RosterSeatCount = 8;
    public const int AlbumItemCount = 24;
    public const int AlbumPerTier = 6;
    public const int EdictCount = 10;
    // B6 fiction gate: no real-dynasty token may enter the roster
    // (mirrors the PalaceSim scan list, U124 fiction line).
    static readonly string[] ForbiddenRosterTokens =
    {
        "Tang", "Song", "Ming", "Qing", "Qin", "Han", "Sui", "Yuan"
    };
    // Frozen wheel face (numbers pack section 1, E4 parity):
    // coins 40 / timber 25 / event 15 / ticket 20, payout bands as in PalaceSim.
    static readonly string[] WheelFrozenOutcomes =
    {
```

```
    WheelCoins, WheelTimber, WheelEventCard, WheelRaidTicket
};
static readonly int[] WheelFrozenWeights = { 40, 25, 15, 20 };
static readonly int[] WheelPayoutMin = { 8, 6, 4, 1 };
static readonly int[] WheelPayoutMax = { 15, 12, 8, 1 };
// Frozen rank ladder (GDD M02): five ranks over four building tiers.
static readonly string[] RankFrozenTiers = { "none", "A03", "A04", "A05", "A06" };
const int RankFrozenHallsPerRank = 3;
const string RanksCurveLocked = "locked_s1_c1017";
public static WheelFile LoadWheel() { return LoadFile&lt;WheelFile&gt;("cfg_g18_wheel"); }
public static RanksFile LoadRanks() { return LoadFile&lt;RanksFile&gt;("cfg_g18_ranks"); }
public static RosterFile LoadRoster() { return LoadFile&lt;RosterFile&gt;("cfg_g18_roster"); }
public static AlbumFile LoadAlbum() { return LoadFile&lt;AlbumFile&gt;("cfg_g18_album"); }
public static EdictsFile LoadEdicts() { return LoadFile&lt;EdictsFile&gt;("cfg_g18_events"); }
static T LoadFile&lt;T&gt;(string name) where T : class
{
    var asset = Resources.Load&lt;TextAsset&gt;(name);
    if (asset == null)
    {
        Logger.Warn("G18.Catalog", name + ".json not found under Resources");
        return null;
    }
    var file = JsonUtility.FromJson&lt;T&gt;(asset.text);
    if (file == null)
    {
        Logger.Warn("G18.Catalog", name + ".json failed to parse");
        return null;
    }
    return file;
}
// ---- lookups (unknown = null, never an error) ----
public static WheelSegment WheelById(WheelFile file, string id)
{
    return RowById(file == null ? null : file.segments, id, r =&gt; r.id);
}
public static RankRow RankById(RanksFile file, string id)
{
    return RowById(file == null ? null : file.ranks, id, r =&gt; r.id);
}
public static RosterRow RosterById(RosterFile file, string id)
{
    return RowById(file == null ? null : file.seats, id, r =&gt; r.id);
}
public static AlbumRow AlbumById(AlbumFile file, string id)
{
    return RowById(file == null ? null : file.items, id, r =&gt; r.id);
}
public static EdictRow EdictById(EdictsFile file, string id)
{
    return RowById(file == null ? null : file.events, id, r =&gt; r.id);
```

```
}
static T RowById&lt;T&gt;(T[] rows, string id, Func&lt;T, string&gt; keyOf) where T : class
{
    if (string.IsNullOrEmpty(id) || rows == null) return null;
    for (int i = 0; i &lt; rows.Length; i++)
    {
        if (keyOf(rows[i]) == id) return rows[i];
    }
    return null;
}
// ---- validators (empty problem list = well-formed) ----
public static List&lt;string&gt; ValidateWheel(WheelFile file)
{
    var problems = new List&lt;string&gt;();
    if (file == null || file.segments == null || file.segments.Length == 0)
    {
        problems.Add("wheel table missing or empty");
        return problems;
    }
    if (file.segments.Length != 4)
        problems.Add("wheel must carry exactly 4 faces (numbers pack section 1), got " + file.segments.Length);
    int sum = 0;
    var ids = new HashSet&lt;string&gt;();
    for (int i = 0; i &lt; file.segments.Length; i++)
    {
        var s = file.segments[i];
        string tag = "wheel[" + i + "]";
        if (string.IsNullOrEmpty(s.id)) problems.Add(tag + " missing id");
        else if (!ids.Add(s.id)) problems.Add(tag + " duplicate id " + s.id);
        sum += s.weight;
        if (i &lt; 4)
        {
            if (s.outcome != WheelFrozenOutcomes[i])
                problems.Add(tag + " outcome must stay in the frozen order " + WheelFrozenOutcomes[i] + ", got "
            if (s.weight != WheelFrozenWeights[i])
                problems.Add(tag + " weight must stay at the frozen " + WheelFrozenWeights[i] + " (numbers pack
            if (s.payoutMin != WheelPayoutMin[i] || s.payoutMax != WheelPayoutMax[i])
                problems.Add(tag + " payout band must stay at the E4 band [" + WheelPayoutMin[i] + "," + WheelPa
        }
        if (string.IsNullOrEmpty(s.nameKey)) problems.Add(tag + " missing nameKey");
    }
    if (sum != 100) problems.Add("wheel weights must sum to 100, got " + sum);
    return problems;
}
public static List&lt;string&gt; ValidateRanks(RanksFile file)
{
    var problems = new List&lt;string&gt;();
    if (file == null || file.ranks == null || file.ranks.Length == 0)
    {
        problems.Add("ranks table missing or empty");
```

```
            return problems;
        }
        if (file.ranks.Length != RankCount)
            problems.Add("rank ladder must hold exactly " + RankCount + " steps (GDD M02), got " + file.ranks.Length
        if (file.curveStatus != RanksCurveLocked)
            problems.Add("curveStatus must record the locked state '" + RanksCurveLocked + "' (numbers pack section
        if (file.hallSlotCap != PalaceSim.HallSlotCap)
            problems.Add("hallSlotCap must mirror PalaceSim.HallSlotCap " + PalaceSim.HallSlotCap + " (full palace =
        if (file.hallCoinCurve == null || file.hallCoinCurve.Length != PalaceSim.HallSlotCap)
            problems.Add("hallCoinCurve must hold exactly " + PalaceSim.HallSlotCap + " per-slot costs (one per hall
        else
        {
            for (int i = 0; i &lt; file.hallCoinCurve.Length; i++)
            {
                if (file.hallCoinCurve[i] != 8 + i / 5)
                    problems.Add("hallCoinCurve[" + i + "] must stay at the locked v0 ladder 8+slot/5 = " + (8 + i /
            }
        }
        var ids = new HashSet&lt;string&gt;();
        for (int i = 0; i &lt; file.ranks.Length; i++)
        {
            var r = file.ranks[i];
            string tag = "rank[" + i + "]";
            if (string.IsNullOrEmpty(r.id)) problems.Add(tag + " missing id");
            else if (!ids.Add(r.id)) problems.Add(tag + " duplicate id " + r.id);
            if (string.IsNullOrEmpty(r.nameKey)) problems.Add(tag + " missing nameKey");
            if (i &lt; RankFrozenTiers.Length &amp;&amp; r.buildingTier != RankFrozenTiers[i])
                problems.Add(tag + " buildingTier must stay at the frozen " + RankFrozenTiers[i] + " (four hall tier
            if (r.hallsPerRank != RankFrozenHallsPerRank)
                problems.Add(tag + " hallsPerRank must stay at the E4 graybox parity " + RankFrozenHallsPerRank + "
        }
        return problems;
    }
    public static List&lt;string&gt; ValidateRoster(RosterFile file)
    {
        var problems = new List&lt;string&gt;();
        if (file == null || file.seats == null || file.seats.Length == 0)
        {
            problems.Add("roster table missing or empty");
            return problems;
        }
        if (file.seats.Length != RosterSeatCount)
            problems.Add("roster must hold exactly " + RosterSeatCount + " seats (numbers pack section 6), got " + f
        var ids = new HashSet&lt;string&gt;();
        var tokens = new HashSet&lt;string&gt;();
        for (int i = 0; i &lt; file.seats.Length; i++)
        {
            var s = file.seats[i];
            string tag = "roster[" + i + "]";
            if (string.IsNullOrEmpty(s.id)) problems.Add(tag + " missing id");
```

```
            else if (!ids.Add(s.id)) problems.Add(tag + " duplicate id " + s.id);
            if (string.IsNullOrEmpty(s.token)) problems.Add(tag + " missing token");
            else
            {
                if (!tokens.Add(s.token)) problems.Add(tag + " duplicate token " + s.token);
                for (int j = 0; j &lt; ForbiddenRosterTokens.Length; j++)
                {
                    if (s.token.Contains(ForbiddenRosterTokens[j]))
                        problems.Add(tag + " token carries real-dynasty token '" + ForbiddenRosterTokens[j] + "' (B6
                }
            }
            if (string.IsNullOrEmpty(s.nameKey)) problems.Add(tag + " missing nameKey");
        }
        return problems;
    }
    public static List&lt;string&gt; ValidateAlbum(AlbumFile file)
    {
        var problems = new List&lt;string&gt;();
        if (file == null || file.items == null || file.items.Length == 0)
        {
            problems.Add("album table missing or empty");
            return problems;
        }
        if (file.items.Length != AlbumItemCount)
            problems.Add("album must hold exactly " + AlbumItemCount + " curios (four tiers x " + AlbumPerTier + "),
        var ids = new HashSet&lt;string&gt;();
        var stems = new HashSet&lt;string&gt;();
        int bronze = 0, silver = 0, gold = 0, jade = 0;
        for (int i = 0; i &lt; file.items.Length; i++)
        {
            var a = file.items[i];
            string tag = "album[" + i + "]";
            if (string.IsNullOrEmpty(a.id)) problems.Add(tag + " missing id");
            else if (!ids.Add(a.id)) problems.Add(tag + " duplicate id " + a.id);
            if (a.tier == TierBronze) bronze++;
            else if (a.tier == TierSilver) silver++;
            else if (a.tier == TierGold) gold++;
            else if (a.tier == TierJade) jade++;
            else problems.Add(tag + " unknown tier " + a.tier);
            if (string.IsNullOrEmpty(a.stem)) problems.Add(tag + " missing stem");
            else if (!stems.Add(a.stem)) problems.Add(tag + " duplicate stem " + a.stem);
            if (string.IsNullOrEmpty(a.nameKey)) problems.Add(tag + " missing nameKey");
        }
        if (bronze != AlbumPerTier) problems.Add("bronze tier must hold " + AlbumPerTier + " items, got " + bronze);
        if (silver != AlbumPerTier) problems.Add("silver tier must hold " + AlbumPerTier + " items, got " + silver);
        if (gold != AlbumPerTier) problems.Add("gold tier must hold " + AlbumPerTier + " items, got " + gold);
        if (jade != AlbumPerTier) problems.Add("jade tier must hold " + AlbumPerTier + " items, got " + jade);
        return problems;
    }
    public static List&lt;string&gt; ValidateEdicts(EdictsFile file)
```

```
        {
            var problems = new List&lt;string&gt;();
            if (file == null || file.events == null || file.events.Length == 0)
            {
                problems.Add("edict table missing or empty");
                return problems;
            }
            if (file.events.Length != EdictCount)
                problems.Add("edict table must hold exactly " + EdictCount + " events (numbers pack section 1), got " +
            var ids = new HashSet&lt;string&gt;();
            int positive = 0, absurd = 0;
            for (int i = 0; i &lt; file.events.Length; i++)
            {
                var e = file.events[i];
                string tag = "edict[" + i + "]";
                if (string.IsNullOrEmpty(e.id)) problems.Add(tag + " missing id");
                else if (!ids.Add(e.id)) problems.Add(tag + " duplicate id " + e.id);
                if (e.polarity == PolarityPositive) positive++;
                else if (e.polarity == PolarityAbsurd) absurd++;
                else problems.Add(tag + " polarity must be positive/absurd, got " + e.polarity);
                if (e.bonusMin &lt; 4 || e.bonusMax &gt; 8 || e.bonusMin &gt; e.bonusMax)
                    problems.Add(tag + " bonus must stay inside the E4 band [4,8] with min&lt;=max, got [" + e.bonusMin
                if (string.IsNullOrEmpty(e.textKey)) problems.Add(tag + " missing textKey");
            }
            if (positive == 0) problems.Add("edict pool must keep at least one positive face (comedy dual state)");
            if (absurd == 0) problems.Add("edict pool must keep at least one absurd face (comedy dual state)");
            return problems;
        }
    }
}
// G18Sfx.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.Palace
{
    /// &lt;summary&gt;
    /// Procedural sfx + signature hook for G18 (task S1-T006, X028 earworm
    /// red line: hook hummable within 3s, unique across the portfolio, zero
    /// shipped audio files - pure math over float[] buffers, deterministic,
    /// EditMode testable, no Random, no files; G17Sfx family paradigm).
    ///
    /// G18 signature hook "gong bloom + ka-da x3 + coin ding": a low
    /// inharmonic palace-gong bloom opens (the portfolio's only gong
    /// voice - taskbook mutex check: G02 star chime RINGS UP bright, G03
    /// jelly wobbles in three steps, G06 glides then knocks, G16 climbs a
    /// rising money line, G17 marimba is a struck FALLING PAIR), the wheel
    /// ratchet answers in a tightening triple (the exact calibrated tick
    /// the wheel already plays per segment), and a bright coin double-ding
```

```
/// lands the payoff - the hook promises exactly what the game plays.
/// The same recipes feed the core feedback trio (resource coin/timber
/// dual timbre, build hammer knock-nail-solid), so the signature is
/// heard organically inside the opening. The AU02 bgm row (dizi flute
/// melody + guzheng pluck + light drum, MCP face) rides the reserved
/// loop slot; the melody seat keeps the portfolio's only flute voice.
/// &lt;/summary&gt;
public static class G18Sfx
{
    public const int SampleRate = 22050;
    public const string SpinTick = "spin_tick";       // wheel ratchet click (per-segment ka-da)
    public const string GongBloom = "gong_bloom";     // palace gong opening bloom (the signature open)
    public const string ResourceCoin = "resource_coin";   // silver coin double-ding (bright metal)
    public const string ResourceTimber = "resource_timber"; // timber load soft wooden knock (dark)
    public const string BuildHammer = "build_hammer"; // build beat: knock - nail - solid (three)
    public const string Leitmotif = "leitmotif";      // signature hook: gong + ka-da x3 + coin ding
    public const string RaidHaul = "raid_haul";         // S3-T004 raid-report beat: spoils land, an ascending clink
    public const string RaidCounter = "raid_counter";   // S3-T004 raid-report beat: the gong sags low (comedic defl
    // ---- delivery caliber (G17Sfx family law, zero-file family) -------
    // Same-batch loudness spread &lt;= 1 dB measured K-weighted (BS.1770),
    // sample peak &lt;= -3.6 dBFS so true peak keeps the -3 dB spec with
    // inter-sample overshoot margin; the -16 LUFS anchor belongs to the
    // AU02 bgm row (MCP face), not to these short acts.
    public const double SamplePeakCap = 0.66;     // -3.6 dBFS sample peak
    public const double FamilySpreadMaxDb = 1.0;  // same-batch loudness difference &lt;= 1 dB
    // signature phrase gaps: the ratchet triple tightens into the payoff
    public const double LeitmotifGap1Sec = 0.10;   // gong bloom .. first ka-da (breath)
    public const double LeitmotifTickGap1Sec = 0.055; // ka-da .. ka-da
    public const double LeitmotifTickGap2Sec = 0.045; // ka-da .. ka-da (tightest, then coin)
    public const double LeitmotifGap2Sec = 0.09;   // last ka-da .. coin ding
    // ---- AU02 bgm work-order face (MCP seat, TJ custom only - U122) ----
    public const string BgmMelodyTimbre = "dizi_flute";            // the portfolio's only flute melody voice
    public const string BgmAccompTimbre = "guzheng_pluck_light_drum"; // accompaniment seat, never the melody
    /// &lt;summary&gt;Portfolio hook timbre registry (declared face, source =
    /// taskbook S1-T006 mutex check): each sibling hook owns a distinct
    /// signature voice; the registry is mechanically asserted duplicate-
    /// free so the declared mutex stays pinned in code.&lt;/summary&gt;
    public static readonly string[] PortfolioHookTimbres =
    {
        "g02_star_chime_rising",     // G02: chime rings up (bright)
        "g03_jelly_three_fall",      // G03: jelly wobble, three steps
        "g06_glide_then_knock",      // G06: glide answered by a knock
        "g16_rising_money_three",    // G16: rising three-note money line
        "g17_marimba_fall_pair",     // G17: struck falling marimba pair
        "g18_palace_gong_bloom",     // G18: low inharmonic gong bloom (this game)
    };
    static readonly Dictionary&lt;string, AudioClip&gt; ClipCache = new Dictionary&lt;string, AudioClip&gt;();
    static readonly Dictionary&lt;string, float[]&gt; _calibratedActs = new Dictionary&lt;string, float[]&gt;();
    static bool _familySettled;
    static AudioSource _host;   // sfx channel: fire-and-forget one-shots
```

```
{
    int g = 0;
    while (sim.Timber &lt; flow.NextTimberCost &amp;&amp; g++ &lt; 50)
    {
        sim.ForceNextSpin(PalaceSim.WheelOutcome.Timber);
        sim.SpinWheel();
    }
}
[Test]
public void Open_Readouts_Grid_Cost_Band_And_Rank_Progress()
{
    var sim = new PalaceSim(12345);
    var flow = new HallFlow(sim);
    Assert.AreEqual(0, flow.Halls);
    Assert.AreEqual(8, flow.NextCoinCost, "v0 curve: 8 + Halls/5 at zero halls");
    Assert.AreEqual(3, flow.NextTimberCost, "HallTimberCost E4 anchor");
    Assert.AreEqual("none", flow.HallBandId, "rank_1 carries no building tier yet");
    Assert.AreEqual(1, flow.Rank);
    Assert.AreEqual(3, flow.HallsPerRank, "frozen hallsPerRank parity");
    Assert.AreEqual(3, flow.HallsUntilNextRank, "three halls to the first promotion");
    Assert.IsTrue(flow.CanBuild, "the opening purse (50/10) affords the first hall");
    Assert.AreEqual(0, flow.CoinGap);
    Assert.AreEqual(0, flow.TimberGap);
    Assert.IsFalse(flow.RescueEmbedDue, "no gap = no embed");
    Assert.AreEqual(0, flow.IdleCoinsEarned);
    Assert.AreEqual(0f, flow.IdleRatePerSec, "zero halls earn nothing");
    Assert.AreEqual(0, flow.OfflineSummaryDay);
    Assert.AreEqual(0, flow.CoinLedger);
    Assert.AreEqual(0, flow.TimberLedger);
}
[Test]
public void Build_RoundTrip_Ledger_Parity_And_Rank_Promotion()
{
    var sim = new PalaceSim(2024);
    var flow = new HallFlow(sim);
    for (int i = 0; i &lt; 3; i++)
    {
        Fund(flow, sim);
        Assert.IsTrue(flow.Build(), "funded build #" + i + " must land");
    }
    Assert.AreEqual(3, flow.Halls);
    Assert.AreEqual(3, flow.BuildCount);
    // round-trip accounting: BuildCount x cost allocation = HallSpend + TimberSpend
    Assert.AreEqual(sim.HallSpend, flow.CoinLedger, "presenter ledger == sim coin spend, zero drift");
    Assert.AreEqual(sim.TimberSpend, flow.TimberLedger, "presenter ledger == sim timber spend, zero drift");
    Assert.AreEqual(3 * PalaceSim.HallTimberCost, sim.TimberSpend);
    Assert.AreEqual(3, sim.CountEvents("build"));
    // rank promotion readout: every 3 halls promote one rank
    Assert.AreEqual(2, flow.Rank);
    Assert.AreEqual("A03", flow.HallBandId, "rank_2 promotes the hall band to A03");
```

```
    Assert.AreEqual(1, sim.CountEvents("rank_up"));
    Assert.AreEqual(3, flow.HallsUntilNextRank, "fresh period after the promotion");
}
[Test]
public void Band_Ladder_Advances_With_Rank()
{
    var sim = new PalaceSim(31);
    var flow = new HallFlow(sim);
    for (int i = 0; i &lt; 6; i++)
    {
        Fund(flow, sim);
        Assert.IsTrue(flow.Build(), "funded build #" + i + " must land");
    }
    Assert.AreEqual(6, flow.Halls);
    Assert.AreEqual(3, flow.Rank, "two promotions over six halls");
    Assert.AreEqual("A04", flow.HallBandId, "rank_3 band = A04");
    Assert.AreEqual(2, sim.CountEvents("rank_up"));
    Assert.AreEqual(3, flow.HallsUntilNextRank, "three more halls to rank 4");
}
[Test]
public void Idle_Determinism_Two_Sims_Tick_Equal_And_Day_Port()
{
    var a = new PalaceSim(7);
    var b = new PalaceSim(7);
    var fa = new HallFlow(a);
    var fb = new HallFlow(b);
    foreach (var flow in new[] { fa, fb })
    {
        Assert.IsTrue(flow.Build());
        Assert.IsTrue(flow.Build());
        Assert.AreEqual(2, flow.Halls);
    }
    Assert.AreEqual(2 * PalaceSim.IdleCoinsPerHallPerSec, fa.IdleRatePerSec);
    // determinism assertion: the same dt sequence on two sims pays equal
    // idle income tick by tick (AdvanceClock stays the single driver).
    float[] dts = { 0.5f, 0.5f, 1f, 2f, 0.3f, 3f };
    foreach (float dt in dts)
    {
        a.AdvanceClock(dt);
        b.AdvanceClock(dt);
        Assert.AreEqual(a.IdleEarned, b.IdleEarned, "idle income parity per tick");
        Assert.AreEqual(fa.IdleCoinsEarned, fb.IdleCoinsEarned, "flow readout parity per tick");
    }
    // 2 halls x 1.2/s x 7.3s = 17.52 -&gt; integer ledger 17 (exact, deterministic)
    Assert.AreEqual(17, fa.IdleCoinsEarned);
    Assert.AreEqual(17, a.IdleEarned);
    Assert.AreEqual(0, fa.OfflineSummaryDay);
    // offline-summary data port: the DayIndexOf window rolls over
    a.AdvanceClock(86405f);
    Assert.AreEqual(1, fa.OfflineSummaryDay, "day port reads the sim clock, never wall time");
```

```
            Assert.GreaterOrEqual(a.CountEvents("day_rollover"), 1);
            Assert.Greater(fa.IdleCoinsEarned, 17, "idle keeps accruing through the port window");
        }
        [Test]
        public void NearMiss_TimberGap_Band_Triggers_Embed_And_Refuse_Event()
        {
            var sim = new PalaceSim(11);
            var flow = new HallFlow(sim);
            // pure start purse, zero funding: three builds drain timber to 1
            Assert.IsTrue(flow.Build());
            Assert.IsTrue(flow.Build());
            Assert.IsTrue(flow.Build());
            Assert.IsFalse(flow.CanBuild, "timber 1 &lt; 3 refuses the fourth hall");
            Assert.AreEqual(0, flow.CoinGap, "coins still cover the cost");
            Assert.AreEqual(2, flow.TimberGap, "two timber short");
            Assert.IsTrue(flow.RescueEmbedDue, "small gap = the near-miss embed point (rescue seat data face)");
            Assert.IsFalse(flow.Build(), "refuse rides the sim event stream");
            Assert.AreEqual(3, flow.BuildCount, "the refused build does not count");
            Assert.AreEqual(1, sim.CountEvents("op_rejected"), "the broke reject is the embed host event");
            Assert.AreEqual(flow.CoinLedger, sim.HallSpend);
        }
        [Test]
        public void LargeCoinGap_Outside_Embed_Band()
        {
            var sim = new PalaceSim(11);
            var flow = new HallFlow(sim);
            // drain: timber force-funded (timber spins never pay coins), coins never
            // funded - the purse burns 8+8+8+8+8+9 = 49 of the opening 50. Funding
            // runs BEFORE the affordability check or a timber-short purse fakes a
            // coin-broke stop.
            int guard = 0;
            while (guard++ &lt; 30)
            {
                FundTimberOnly(flow, sim);
                if (!flow.CanBuild) break;
                Assert.IsTrue(flow.Build(), "funded build must land");
            }
            Assert.Less(guard, 30, "drain must converge");
            Assert.AreEqual(6, flow.Halls);
            Assert.AreEqual(1, sim.Coins, "deterministic coin floor after the drain");
            Assert.AreEqual(8, flow.CoinGap, "cost 9 (Halls 6) - purse 1");
            Assert.Greater(flow.CoinGap, HallFlow.NearMissCoinGapMax, "far-from-afford purse");
            Assert.IsFalse(flow.RescueEmbedDue, "outside the band the embed stays armed off");
            Assert.IsFalse(flow.Build());
            Assert.AreEqual(1, sim.CountEvents("op_rejected"));
            Assert.AreEqual(6, flow.BuildCount);
        }
    }
}
// PalaceHookFlowTests.cs
```

```
using System;
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S1-T006 slice b acceptance: the scripted hook timeline must reproduce the
    // E4 graybox red-line-11 envelope (numbers-pack section 8 / docs/05 -
    // thresholds never widened): H1 caption 0.00s, H2 first op &lt;=3s, H3 first
    // resource drop &lt;=1s after the op, H4 three core actions inside 10s, H5
    // first raid report comedy reversal &lt;=10s in BOTH verdict states with the
    // share-code self-spread seat, and the opening leaves the ring playable.
    public class PalaceHookFlowTests
    {
        static string FieldOf(PalaceSim sim, string eventName, string key)
        {
            for (int i = 0; i &lt; sim.Events.Count; i++)
            {
                if (sim.Events[i].Name != eventName) continue;
                string token = key + "=";
                int at = sim.Events[i].Detail.IndexOf(token, StringComparison.Ordinal);
                if (at &lt; 0) continue;
                int start = at + token.Length;
                int end = start;
                while (end &lt; sim.Events[i].Detail.Length &amp;&amp; sim.Events[i].Detail[end] != ' ') end++;
                return sim.Events[i].Detail.Substring(start, end - start);
            }
            return "";
        }
        [Test]
        public void HookTimeline_RedLine11_Matches_E4_Parity()
        {
            var sim = new PalaceSim(2026);
            PalaceHookFlow.Run(sim);
            float captionT = sim.FirstEventTime("caption");
            Assert.AreEqual(0f, captionT, 1e-6f, "H1: caption lands at t=0 (no white-screen teaching)");
            float firstOpT = sim.FirstEventTime("spin");
            Assert.LessOrEqual(firstOpT, 3f, "H2: first operation (wheel tap) &lt;=3s");
        }
        [Test]
        public void HookTimeline_H3_FirstResource_Within1s_AfterFirstOp()
        {
            var sim = new PalaceSim(2026);
            PalaceHookFlow.Run(sim);
            float firstOpT = sim.FirstEventTime("spin");
            float resourceT = sim.FirstEventTime("resource_gain");
            Assert.LessOrEqual(resourceT - firstOpT, 1f, "H3: first resource drop &lt;=1s after the first op");
            Assert.LessOrEqual(resourceT, 1f, "H3: first resource lands inside the hook window head");
        }
        [Test]
        public void HookTimeline_H4_ThreeCoreActions_Inside10s()
```

```
        {
            var sim = new PalaceSim(2026);
            PalaceHookFlow.Run(sim);
            int core = 0;
            for (int i = 0; i &lt; sim.Events.Count; i++)
            {
                var e = sim.Events[i];
                if (e.Time &gt; 10f) break;
                if (e.Name == "spin" || e.Name == "build" || e.Name == "raid_report") core++;
            }
            Assert.GreaterOrEqual(core, 3, "H4: three core actions inside the 10s window, got " + core);
        }
        [Test]
        public void HookTimeline_H5_FirstReport_ComedyReversion_BothStates()
        {
            var haul = new PalaceSim(2026);
            PalaceHookFlow.Run(haul, 1f);
            float reportT = haul.FirstEventTime("raid_report");
            Assert.LessOrEqual(reportT, 10f, "H5: first battle report &lt;=10s");
            Assert.AreEqual("haul", FieldOf(haul, "raid_report", "verdict"), "scripted haul state fires");
            Assert.AreNotEqual("", FieldOf(haul, "raid_report", "comedyclause"), "haul state carries the comedy clause")
            Assert.AreEqual(1, haul.CountEvents("share_code"), "the first report carries its share code (self-spread sea
            var counter = new PalaceSim(2026);
            PalaceHookFlow.Run(counter, 0f);
            Assert.AreEqual("countersteal", FieldOf(counter, "raid_report", "verdict"),
                "counter-steal state is reachable (dual-state reversal, zero fail states)");
            Assert.AreNotEqual("", FieldOf(counter, "raid_report", "comedyclause"), "counter-steal state carries the com
        }
        [Test]
        public void HookTimeline_Leaves_Ring_Playable()
        {
            var sim = new PalaceSim(2026);
            PalaceHookFlow.Run(sim);
            var wheel = new WheelFlow(sim);
            Assert.IsTrue(wheel.CanSpinNow, "the wheel stays ready for the first real tap (zero threshold)");
            Assert.Greater(wheel.NaturalSpinsLeftToday, 0, "natural spin budget intact after the scripted timeline");
            var hall = new HallFlow(sim);
            Assert.AreEqual(1, hall.Halls, "one hall built by the timeline");
            Assert.Less(hall.Halls, hall.SlotCap, "the hall grid still has room after the timeline");
        }
    }
}
// PalaceOfflineSummaryFlowTests.cs
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S2-T003 slice-a acceptance (taskbook: cross-day window three-source
    // readout parity - the OfflineSummaryDay/DayIndexOf data port the
    // S1-T003 slice-a reserved, redeemed). The window anchor stamps the
```

```
// last-seen face; the summary then parses the in-window event stream
// (robbery stolen sums, revenge verdict split) against the sim ledgers
// (idle delta off the anchor, theft delta parity). Verdicts ride the
// seed + the SetHaulChance seam; the revenge roll stays the sim's own
// 0.80 chance so the assertions stay verdict-agnostic and pin the
// presenter-mirror parity instead (RevengeFlow ledger round-trip law).
public class PalaceOfflineSummaryFlowTests
{
    static void GrantTickets(PalaceSim sim, int n)
    {
        int g = 0;
        while (sim.RaidTickets &lt; n &amp;&amp; g++ &lt; 50)
        {
            sim.ForceNextSpin(PalaceSim.WheelOutcome.RaidTicket);
            Assert.IsTrue(sim.SpinWheel(), "forced ticket spin must land");
        }
        Assert.Less(g, 50, "ticket funding must converge");
    }
    [Test]
    public void Cross_Day_Window_Three_Source_Parity()
    {
        var sim = new PalaceSim(20260925);
        var flow = new OfflineSummaryFlow(sim);
        var anchor = flow.AnchorNow();
        Assert.AreEqual(0, anchor.IdleEarned, "anchor stamps the zero-idle birth face");
        Assert.IsTrue(sim.BuildHall(), "one hall banks the idle face");
        // robbery face: haul -&gt; the 4-10s retaliation lands a being_raided
        sim.SetHaulChance(1f);
        GrantTickets(sim, 1);
        Assert.IsTrue(sim.RaidOpponent());
        sim.AdvanceClock(20f);
        Assert.Greater(sim.TheftLoss, 0, "the retaliation stole something");
        // revenge face: consume the deterministic code (verdict stays the
        // sim's own roll; the RevengeFlow mirror carries the parity)
        var revenge = new RevengeFlow(sim);
        Assert.IsTrue(sim.RevengePending, "the robbery armed the revenge seat");
        Assert.IsTrue(revenge.EnterCode(PalaceSim.RevengeCodeOf(sim.RevengeTarget)));
        // cross the day boundary: the absence window spans into tomorrow
        sim.AdvanceClock(86400f);
        var snap = flow.Summarize(anchor);
        Assert.AreEqual(1, snap.AbsenceDays, "the window crossed exactly one day boundary (DayIndexOf port)");
        Assert.AreEqual(1, snap.RobbedCount, "one in-window robbery");
        Assert.AreEqual(sim.TheftLoss, snap.RobbedLoss,
            "event-stream stolen parse == theft ledger (anchor predates every robbery)");
        Assert.AreEqual(1, snap.RevengeCount, "one in-window revenge");
        Assert.AreEqual(1, snap.RevengeWins + snap.RevengeLosses, "exactly one verdict");
        Assert.AreEqual(revenge.RevengeLootLedger, snap.RevengeLoot,
            "revenge loot parity against the RevengeFlow mirror (round-trip law)");
        Assert.AreEqual(revenge.RevengeLossLedger, snap.RevengeLoss,
            "revenge loss parity against the RevengeFlow mirror");
```

```
    Assert.Greater(snap.IdleEarnedDuring, 0, "the hall banked idle income across the window");
    Assert.AreEqual(sim.IdleEarned, snap.IdleEarnedDuring,
        "idle ledger delta parity (anchor stamped at zero idle)");
    Assert.AreEqual(OfflineSummaryFlow.ToneSelfMock, snap.ComedyTone,
        "a robbed window reads the self-mocking face");
}
[Test]
public void Quiet_Window_Keeps_Decree_Tone_And_Zero_Robbery_Faces()
{
    var sim = new PalaceSim(7);
    var flow = new OfflineSummaryFlow(sim);
    var anchor = flow.AnchorNow();
    Assert.IsTrue(sim.BuildHall());
    sim.AdvanceClock(90000f);   // a quiet day-and-a-bit away
    var snap = flow.Summarize(anchor);
    Assert.AreEqual(1, snap.AbsenceDays);
    Assert.AreEqual(0, snap.RobbedCount, "no robbery in a quiet window");
    Assert.AreEqual(0, snap.RobbedLoss);
    Assert.AreEqual(0, snap.RevengeCount);
    Assert.AreEqual(0, snap.RevengeWins + snap.RevengeLosses);
    Assert.AreEqual(sim.IdleEarned, snap.IdleEarnedDuring,
        "the idle face still banks across the quiet window");
    Assert.Greater(snap.IdleEarnedDuring, 0);
    Assert.AreEqual(OfflineSummaryFlow.ToneDecree, snap.ComedyTone,
        "a quiet window reads the decree face");
}
[Test]
public void Multi_Robbery_Window_Aggregates_Both_Thefts()
{
    var sim = new PalaceSim(77);
    var flow = new OfflineSummaryFlow(sim);
    var anchor = flow.AnchorNow();
    sim.SetHaulChance(1f);
    for (int i = 0; i &lt; 2; i++)
    {
        GrantTickets(sim, 1);
        Assert.IsTrue(sim.RaidOpponent(), "raid " + i);
        sim.AdvanceClock(20f);
    }
    var snap = flow.Summarize(anchor);
    Assert.AreEqual(2, snap.RobbedCount, "both in-window robberies aggregate");
    Assert.AreEqual(sim.TheftLoss, snap.RobbedLoss,
        "parsed stolen sum == theft ledger across two robberies");
}
[Test]
public void Anchor_Isolates_Events_Before_The_Stamp()
{
    var sim = new PalaceSim(99);
    var flow = new OfflineSummaryFlow(sim);
    sim.SetHaulChance(1f);
```

```
            // one robbery BEFORE the anchor
            GrantTickets(sim, 1);
            Assert.IsTrue(sim.RaidOpponent());
            sim.AdvanceClock(20f);
            int theftBefore = sim.TheftLoss;
            Assert.Greater(theftBefore, 0, "pre-anchor robbery stole something");
            // the anchor stamps AFTER the first robbery
            var anchor = flow.AnchorNow();
            // one robbery AFTER the anchor
            GrantTickets(sim, 1);
            Assert.IsTrue(sim.RaidOpponent());
            sim.AdvanceClock(20f);
            var snap = flow.Summarize(anchor);
            Assert.AreEqual(1, snap.RobbedCount, "only the post-anchor robbery counts");
            Assert.AreEqual(sim.TheftLoss - theftBefore, snap.RobbedLoss,
                "the pre-anchor theft is excluded from the window");
            Assert.AreEqual(OfflineSummaryFlow.ToneSelfMock, snap.ComedyTone);
        }
    }
}
// PalaceOfflineSummaryViewHostTests.cs
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S2-T003 slice-b acceptance, return-visit popup pure faces
    // (PalaceRankUpCeremonyViewHostTests paradigm): the comedy tone key
    // mirror (robbed = self-mocking, quiet = decree - the flow's ComedyTone
    // token mapped to its i18n key, never invented copy), the line
    // visibility law (zero-count ledger rows stay hidden), the
    // return-visit gate (an absence counts only once the window crossed a
    // day boundary - the OfflineSummaryDay port), and the externalized
    // copy keys (red line 4). The presenter owns zero invented numbers or
    // text; the three-source parity itself is the slice-a EM family.
    public class PalaceOfflineSummaryViewHostTests
    {
        [Test]
        public void ToneKeyOf_RobbedWindowReadsSelfMock_QuietWindowReadsDecree()
        {
            Assert.AreEqual(OfflineSummaryViewHost.ToneSelfMockKey,
                OfflineSummaryViewHost.ToneKeyOf(true),
                "a robbed window reads the self-mocking face (flow ComedyTone mirror)");
            Assert.AreEqual(OfflineSummaryViewHost.ToneDecreeKey,
                OfflineSummaryViewHost.ToneKeyOf(false),
                "a quiet window reads the decree face");
            Assert.AreEqual("self_mock", OfflineSummaryFlow.ToneSelfMock,
                "the flow tone token stays the slice-a frozen constant");
            Assert.AreEqual("decree", OfflineSummaryFlow.ToneDecree);
        }
        [Test]
```

```
    public void ToneCopyFaces_BothDeclaredAndDistinct_AfterLoad()
    {
        G18Text.Load();
        string decree = G18Text.Get(OfflineSummaryViewHost.ToneDecreeKey);
        string selfMock = G18Text.Get(OfflineSummaryViewHost.ToneSelfMockKey);
        StringAssert.IsMatch(".+", decree, "the decree face carries real copy");
        StringAssert.IsMatch(".+", selfMock, "the self-mocking face carries real copy");
        Assert.AreNotEqual(decree, selfMock,
            "the two tone faces must read different copy (the comedy dual state)");
    }
    [Test]
    public void LineShownOf_OnlyPositiveCountsShowALedgerRow()
    {
        Assert.IsFalse(OfflineSummaryViewHost.LineShownOf(0),
            "a zero-count face stays hidden - the panel never shows empty rows");
        Assert.IsTrue(OfflineSummaryViewHost.LineShownOf(1));
        Assert.IsTrue(OfflineSummaryViewHost.LineShownOf(2),
            "an aggregated window (two robberies) reads its line");
        Assert.IsTrue(OfflineSummaryViewHost.LineShownOf(103680),
            "the idle banking line shows across a full away day");
    }
    [Test]
    public void IsReturnDue_OnlyCountsWindowsThatCrossedADayBoundary()
    {
        Assert.IsFalse(OfflineSummaryViewHost.IsReturnDue(0f, 0f),
            "the same instant is not an absence");
        Assert.IsFalse(OfflineSummaryViewHost.IsReturnDue(0f, 86399f),
            "a window inside the first day is still today - no popup");
        Assert.IsTrue(OfflineSummaryViewHost.IsReturnDue(0f, 86400f),
            "crossing the day boundary opens the return-visit face (OfflineSummaryDay port)");
        Assert.IsTrue(OfflineSummaryViewHost.IsReturnDue(100f, 86400f + 100f),
            "an anchor mid-day counts the boundary it crossed");
        Assert.IsFalse(OfflineSummaryViewHost.IsReturnDue(86400f, 86400f + 10f),
            "a window re-anchored after the close stays same-day - no re-show");
    }
    [Test]
    public void OfflinePopupI18nKeys_AllPresentAfterLoad()
    {
        G18Text.Load();
        string[] keys =
        {
            OfflineSummaryViewHost.TitleKey, OfflineSummaryViewHost.ToneDecreeKey,
            OfflineSummaryViewHost.ToneSelfMockKey, OfflineSummaryViewHost.RobbedKey,
            OfflineSummaryViewHost.RevengeHaulKey, OfflineSummaryViewHost.RevengeCounterKey,
            OfflineSummaryViewHost.IdleKey, OfflineSummaryViewHost.CloseKey,
        };
        foreach (string k in keys)
            Assert.IsTrue(G18Text.Has(k), "return-visit popup key must be declared and loaded: " + k);
    }
}
```

```
}
// PalaceRaidFlowTests.cs
using System;
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S1-T004 slice-a acceptance (taskbook spec block: EM trio = report
    // double-state determinism / robbed-loss formula parity / rescue embed
    // band). HallFlow paradigm: RaidFlow owns the session readouts the sim
    // does not model, PalaceSim stays behaviorally untouched. Verdicts are
    // forced through the SetHaulChance test seam (1.0 / 0.0), so the
    // loot/loss states are verdict-deterministic; every balance assertion
    // reads the live purse, so the trajectories stay seed-independent.
    public class PalaceRaidFlowTests
    {
        static void GrantTickets(PalaceSim sim, int n)
        {
            int g = 0;
            while (sim.RaidTickets &lt; n &amp;&amp; g++ &lt; 50)
            {
                sim.ForceNextSpin(PalaceSim.WheelOutcome.RaidTicket);
                Assert.IsTrue(sim.SpinWheel(), "forced ticket spin must land");
            }
            Assert.Less(g, 50, "ticket funding must converge");
        }
        static void FundCoins(PalaceSim sim, int target)
        {
            int g = 0;
            while (sim.Coins &lt; target &amp;&amp; g++ &lt; 50)
            {
                sim.ForceNextSpin(PalaceSim.WheelOutcome.Coins);
                Assert.IsTrue(sim.SpinWheel());
            }
            Assert.Less(g, 50, "coin funding must converge");
        }
        [Test]
        public void Open_Readouts_Ticket_Gate_Rotation_And_Double_Deny()
        {
            var sim = new PalaceSim(12345);
            var flow = new RaidFlow(sim);
            Assert.AreEqual(0, flow.Tickets);
            Assert.IsFalse(flow.CanRaid);
            Assert.AreEqual(0, flow.RotationIndex, "day 0");
            Assert.AreEqual(PalaceSim.OpponentRoster[0], flow.OpponentToday,
                "deterministic rotation, presenter never re-rolls");
            Assert.AreEqual(flow.OpponentToday, sim.OpponentOfToday(),
                "presenter rotation == sim rotation, zero drift");
            Assert.IsTrue(flow.RosterFiction, "B6 fiction roster mechanical face");
            // ticketless launch rides the sim reject stream
```

```
        Assert.IsFalse(flow.Raid());
        Assert.AreEqual(1, sim.CountEvents("op_rejected"));
        // double seat: arm once, deny the second while pending
        Assert.IsTrue(flow.RequestDouble());
        Assert.IsTrue(flow.DoublePending);
        Assert.IsFalse(flow.RequestDouble(), "one pending double only");
        Assert.AreEqual(1, sim.CountEvents("raid_double_second_request_denied"));
        // ticket gate opens after a wheel-earned ticket
        GrantTickets(sim, 1);
        Assert.GreaterOrEqual(flow.Tickets, 1);
        Assert.IsTrue(flow.CanRaid);
    }
    [Test]
    public void Report_Double_State_Determinism_Ledger_Round_Trip()
    {
        // haul state: chance 1.0, loot into the purse
        var simA = new PalaceSim(777);
        var flowA = new RaidFlow(simA);
        GrantTickets(simA, 1);
        simA.SetHaulChance(1f);
        int coinsBefore = simA.Coins;
        Assert.IsTrue(flowA.Raid());
        Assert.AreEqual("haul", flowA.LastVerdict);
        Assert.AreEqual(flowA.OpponentToday, flowA.LastTarget, "report carries today's roster face");
        Assert.Greater(flowA.LastLoot, 0);
        Assert.AreEqual(coinsBefore + flowA.LastLoot, simA.Coins, "loot lands in the purse");
        Assert.AreEqual(simA.LootEarned, flowA.LootLedger,
            "presenter loot ledger == sim loot ledger, zero drift");
        Assert.Greater(flowA.LastComedyClause.Length, 0, "comedy clause token rides the report");
        Assert.Greater(flowA.LastShareCode.Length, 0, "share-code data port");
        Assert.AreEqual(0, flowA.LastLoss);
        // countersteal state: chance 0.0, loss out of the purse
        var simB = new PalaceSim(31);
        var flowB = new RaidFlow(simB);
        GrantTickets(simB, 1);
        simB.SetHaulChance(0f);
        int before = simB.Coins;
        Assert.IsTrue(flowB.Raid());
        Assert.AreEqual("countersteal", flowB.LastVerdict);
        Assert.Greater(flowB.LastLoss, 0);
        Assert.AreEqual(before - flowB.LastLoss, simB.Coins, "loss leaves the purse");
        Assert.AreEqual(simB.CounterLoss, flowB.CounterLossLedger,
            "presenter counter ledger == sim counter ledger, zero drift");
        Assert.AreEqual(0, flowB.LastLoot);
        Assert.AreEqual(1, simB.CountEvents("raid_report"));
    }
    [Test]
    public void Countersteal_Loss_Formula_Parity_Boundary_Pair()
    {
        // boundary crossover: opening purse 50 -&gt; 50/10 = 5 == clamp floor
```

```
    var simC = new PalaceSim(9);
    var flowC = new RaidFlow(simC);
    GrantTickets(simC, 1);
    simC.SetHaulChance(0f);
    int before = simC.Coins;
    Assert.AreEqual(PalaceSim.StartPurseCoins, before, "StartPurseCoins anchor");
    Assert.IsTrue(flowC.Raid());
    Assert.AreEqual(Math.Max(5, before / 10), flowC.LastLoss,
        "max(5, Coins/10) parity at the crossover");
    Assert.AreEqual(5, flowC.LastLoss, "crossover reads the clamp floor exactly");
    Assert.AreEqual(simC.CounterLoss, flowC.CounterLossLedger);
    // formula face: a rich purse pays the proportional branch
    var simD = new PalaceSim(9);
    var flowD = new RaidFlow(simD);
    GrantTickets(simD, 1);
    FundCoins(simD, 200);
    simD.SetHaulChance(0f);
    int rich = simD.Coins;
    Assert.GreaterOrEqual(rich, 200);
    Assert.IsTrue(flowD.Raid());
    Assert.AreEqual(Math.Max(5, rich / 10), flowD.LastLoss,
        "proportional branch parity, seed-independent");
    Assert.AreEqual(rich - flowD.LastLoss, simD.Coins);
    Assert.AreEqual(simD.CounterLoss, flowD.CounterLossLedger);
}
[Test]
public void Robbed_Rescue_Embed_One_Gate_Per_Raid_Accept_Decline_States()
{
    var sim = new PalaceSim(2024);
    var flow = new RaidFlow(sim);
    GrantTickets(sim, 3);
    sim.SetHaulChance(1f);
    Assert.IsTrue(flow.Raid());
    // the hauled raid queues the system retaliation; mature it on the clock
    sim.AdvanceClock(20f);
    Assert.AreEqual(1, sim.CountEvents("being_raided"), "robbery fires after the haul");
    Assert.Greater(flow.StolenLast, 0);
    Assert.Greater(flow.RobbedByLast.Length, 0);
    Assert.IsTrue(flow.RescueOfferPending, "robbed -&gt; rescue embed armed (one gate per robbery)");
    Assert.AreEqual(1, sim.CountEvents("rescue_offer"));
    Assert.AreEqual(flow.TheftObserved, sim.TheftLoss, "stolen readout parity");
    // second robbery while the offer pends does not stack a second gate
    Assert.IsTrue(flow.Raid());
    sim.AdvanceClock(20f);
    Assert.AreEqual(2, sim.CountEvents("being_raided"));
    Assert.AreEqual(1, sim.CountEvents("rescue_offer"),
        "one rescue gate per robbery - the pending offer holds");
    Assert.IsTrue(flow.RescueOfferPending);
    Assert.AreEqual(flow.TheftObserved, sim.TheftLoss, "stolen parity after the second robbery");
    // decline keeps the stolen loss (zero-penalty data face)
```

```
    int coinsAtDecline = sim.Coins;
    Assert.IsTrue(flow.DeclineRescue());
    Assert.IsFalse(flow.RescueOfferPending);
    Assert.AreEqual(coinsAtDecline, sim.Coins, "decline charges nothing extra");
    Assert.AreEqual(1, sim.CountEvents("rescue_decline"));
    Assert.AreEqual(0, sim.RescueCount);
    // third robbery re-arms the gate; accept restores exactly
    Assert.IsTrue(flow.Raid());
    sim.AdvanceClock(20f);
    Assert.AreEqual(3, sim.CountEvents("being_raided"));
    Assert.AreEqual(2, sim.CountEvents("rescue_offer"), "gate re-arms per robbery after the decline");
    int stolen = flow.StolenLast;
    int coinsBeforeAccept = sim.Coins;
    Assert.IsTrue(flow.AcceptRescue());
    Assert.AreEqual(coinsBeforeAccept + stolen, sim.Coins,
        "guard fund restores the exact stolen amount");
    Assert.AreEqual(sim.RestoredTotal, flow.RestoredLedger, "restored ledger parity");
    Assert.AreEqual(1, sim.RescueCount);
}
// ---- S3-T003 M08: loot display slot data face ----
[Test]
public void LastCurioHauled_ReadsTheHaulGalleryAdd_CounterstealStaysEmpty()
{
    // haul state: the raid's gallery_add lands inside the op window
    var sim = new PalaceSim(4242);
    var flow = new RaidFlow(sim);
    GrantTickets(sim, 1);
    sim.SetHaulChance(1f);
    Assert.AreEqual("", flow.LastCurioHauled, "no raid yet - the slot starts empty");
    Assert.IsTrue(flow.Raid());
    Assert.AreEqual("haul", flow.LastVerdict);
    Assert.AreEqual(1, sim.CountEvents("gallery_add"),
        "a haul verdict drops exactly one curio into the gallery");
    string curioFromEvent = "";
    for (int i = sim.Events.Count - 1; i &gt;= 0; i--)
    {
        if (sim.Events[i].Name != "gallery_add") continue;
        string d = sim.Events[i].Detail;
        int ix = d.IndexOf("curio=", System.StringComparison.Ordinal);
        if (ix &gt;= 0) curioFromEvent = d.Substring(ix + "curio=".Length);
        break;
    }
    Assert.AreEqual(curioFromEvent, flow.LastCurioHauled,
        "the slot reads the curio the haul's gallery_add emitted");
    StringAssert.StartsWith("CR-", flow.LastCurioHauled, "sim curio id space");
    // countersteal state: no gallery_add in the op window - the honest
    // empty-hands face, never a stale earlier haul
    GrantTickets(sim, 1);
    sim.SetHaulChance(0f);
    Assert.IsTrue(flow.Raid());
```

```
            Assert.AreEqual("countersteal", flow.LastVerdict);
            Assert.AreEqual(1, sim.CountEvents("gallery_add"), "a countersteal drops no curio");
            Assert.AreEqual("", flow.LastCurioHauled,
                "the slot reads empty after a countersteal - no stale trophy");
        }
    }
}
// PalaceRankUpCeremonyViewHostTests.cs
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S2-T001 slice-b acceptance, ceremony-layer pure faces
    // (HallViewHostTests paradigm): the flip ladder (five pips, exactly the
    // celebrated one lit), the queue readout law, the invariant rate format,
    // the externalized copy keys, and the presented beat on the funnel -
    // the presenter owns zero invented numbers or text.
    public class PalaceRankUpCeremonyViewHostTests
    {
        static void Fund(PalaceSim sim)
        {
            int g = 0;
            while (sim.Coins &lt; sim.NextHallCoinCost() &amp;&amp; g++ &lt; 60)
            {
                sim.ForceNextSpin(PalaceSim.WheelOutcome.Coins);
                sim.SpinWheel();
            }
            g = 0;
            while (sim.Timber &lt; PalaceSim.HallTimberCost &amp;&amp; g++ &lt; 60)
            {
                sim.ForceNextSpin(PalaceSim.WheelOutcome.Timber);
                sim.SpinWheel();
            }
            Assert.Less(g, 60, "funding must converge");
        }
        [Test]
        public void IsLitPip_ExactlyTheCelebratedRank_LightsItsOwnPip()
        {
            for (int pip = 1; pip &lt;= 5; pip++)
            {
                Assert.IsTrue(RankUpCeremonyViewHost.IsLitPip(pip, pip),
                    "the celebrated rank lights its own pip (the flip face)");
                for (int other = 1; other &lt;= 5; other++)
                    if (other != pip)
                        Assert.IsFalse(RankUpCeremonyViewHost.IsLitPip(pip, other),
                            "pip " + pip + " stays dim while rank " + other + " is celebrated");
            }
        }
        [Test]
        public void PipAlphaOf_LitFull_DimQuarter()
```

```
{
    Assert.AreEqual(1f, RankUpCeremonyViewHost.PipAlphaOf(true), "the lit pip reads full");
    Assert.AreEqual(0.25f, RankUpCeremonyViewHost.PipAlphaOf(false), "dim pips recede to a quarter");
}
[Test]
public void PendingLineShownOf_OnlyWhenTheQueueCarriesMoreThanOne()
{
    Assert.IsFalse(RankUpCeremonyViewHost.PendingLineShownOf(0), "nothing due: no queue readout");
    Assert.IsFalse(RankUpCeremonyViewHost.PendingLineShownOf(1),
        "a single promotion needs no queue line - the decree itself carries it");
    Assert.IsTrue(RankUpCeremonyViewHost.PendingLineShownOf(2),
        "stacked promotions (long offline window) read the queue");
    Assert.IsTrue(RankUpCeremonyViewHost.PendingLineShownOf(3));
}
[Test]
public void RateTextOf_InvariantOneDecimal()
{
    Assert.AreEqual("0", RankUpCeremonyViewHost.RateTextOf(0f));
    Assert.AreEqual("1.2", RankUpCeremonyViewHost.RateTextOf(1.2f), "one hall idles at the frozen 1.2/s");
    Assert.AreEqual("3.6", RankUpCeremonyViewHost.RateTextOf(3.6f), "three halls idle at 3 x 1.2");
}
[Test]
public void CeremonyI18nKeys_AllPresentAfterLoad()
{
    G18Text.Load();
    string[] keys =
    {
        RankUpCeremonyViewHost.CaptionKey, RankUpCeremonyViewHost.TicketKey,
        RankUpCeremonyViewHost.ContinueKey, RankUpCeremonyViewHost.PendingKey,
        "g18.rank.1.name", "g18.rank.2.name",
    };
    foreach (string k in keys)
        Assert.IsTrue(G18Text.Has(k), "ceremony layer key must be declared and loaded: " + k);
}
[Test]
public void Funnel_PresentedBeat_BooksCelebratedRank_OncePerEmittedPromotion()
{
    var sim = new PalaceSim(2024);
    var flow = new RankUpFlow(sim);
    var funnel = new PalaceAdFunnel(sim);
    for (int i = 0; i &lt; 3; i++)
    {
        Fund(sim);
        Assert.IsTrue(sim.BuildHall(), "funded build must land");
    }
    Assert.IsTrue(flow.CeremonyDue, "the emitted promotion is one pending ceremony");
    Assert.AreEqual(2, flow.NextCeremonyRank);
    // the host law under test: read the celebrated rank BEFORE draining,
    // book the beat, then MarkPresented - one beat per emitted promotion
    int rank = flow.NextCeremonyRank;
```

```
            flow.MarkPresented();
            funnel.ReportRankUpCeremony(rank);
            Assert.AreEqual(1, funnel.CountOf(PalaceEventIds.RankUpCeremony), "the presented beat lands once");
            Assert.IsTrue(funnel.Funnel[funnel.IndexOf(PalaceEventIds.RankUpCeremony)].Contains("rank=2"),
                "the beat carries the celebrated rank payload");
            // a drained ceremony is no longer staged: the host gate (TestContinue
            // returns false when nothing is shown) keeps the beat at one per
            // emitted promotion, never one per re-tap
            Assert.IsFalse(flow.CeremonyDue, "presented caught up with emitted");
            flow.MarkPresented();
            Assert.AreEqual(1, funnel.CountOf(PalaceEventIds.RankUpCeremony),
                "no re-book: the beat rode the staged ceremony, and nothing is staged");
        }
    }
}
// PalaceRankUpFlowTests.cs
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S2-T001 slice-a acceptance (EM: rank_up event flow -&gt; ceremony state
    // readout parity, the presenter never re-rolls). HallFlow test paradigm:
    // forced spins only, every trajectory rng-free on the coin face, so the
    // readouts are seed-independent and deterministic; two presenters over
    // one sim read identical ceremony state.
    public class PalaceRankUpFlowTests
    {
        static void Fund(PalaceSim sim)
        {
            int g = 0;
            while (sim.Coins &lt; sim.NextHallCoinCost() &amp;&amp; g++ &lt; 60)
            {
                sim.ForceNextSpin(PalaceSim.WheelOutcome.Coins);
                sim.SpinWheel();
            }
            g = 0;
            while (sim.Timber &lt; PalaceSim.HallTimberCost &amp;&amp; g++ &lt; 60)
            {
                sim.ForceNextSpin(PalaceSim.WheelOutcome.Timber);
                sim.SpinWheel();
            }
            Assert.Less(g, 60, "funding must converge");
        }
        [Test]
        public void NoPromotion_CeremonyStaysClosed_And_PresentIsNoOp()
        {
            var sim = new PalaceSim(12345);
            var flow = new RankUpFlow(sim);
            Assert.AreEqual(0, flow.RankUpCount);
            Assert.IsFalse(flow.CeremonyDue);
```

```
    Assert.AreEqual(0, flow.PendingCount);
    Assert.AreEqual(1, flow.Rank);
    Assert.AreEqual(1, flow.NextCeremonyRank, "no pending event: reads the live rank, never invents one");
    Assert.AreEqual("none", flow.CeremoniedBadgeTier, "rank_1 carries no building tier yet");
    Assert.AreEqual(0f, flow.ExpandedIdleRatePerSec, "zero halls earn nothing");
    Assert.AreEqual(0, flow.PresentedCount);
    flow.MarkPresented();
    Assert.AreEqual(0, flow.PresentedCount, "presenting with nothing due is a no-op");
    Assert.IsFalse(flow.CeremonyDue);
    // S2 registry expansion: the ceremony + unlock beats joined the family
    CollectionAssert.Contains(PalaceEventIds.All, PalaceEventIds.RankUpCeremony);
    CollectionAssert.Contains(PalaceEventIds.All, PalaceEventIds.AchievementUnlock);
    Assert.AreEqual(14, PalaceEventIds.All.Length, "12 ids + 2 S2 system ids");
}
[Test]
public void ThirdHallLands_CeremonyDue_Readouts_Parity_NoReroll()
{
    var sim = new PalaceSim(2024);
    var flow = new RankUpFlow(sim);
    for (int i = 0; i &lt; 3; i++)
    {
        Fund(sim);
        Assert.IsTrue(sim.BuildHall(), "funded build must land");
    }
    Assert.AreEqual(1, sim.CountEvents("rank_up"));
    Assert.IsTrue(flow.CeremonyDue, "the emitted promotion is exactly one pending ceremony");
    Assert.AreEqual(1, flow.PendingCount);
    Assert.AreEqual(2, flow.NextCeremonyRank, "celebrated rank parsed off the rank=N event detail");
    Assert.AreEqual("A03", flow.CeremoniedBadgeTier, "badge flips to the rank_2 tier band");
    Assert.AreEqual(3, flow.CeremoniedHallsPerRank, "frozen hallsPerRank parity, zero drift");
    Assert.AreEqual(3 * PalaceSim.IdleCoinsPerHallPerSec, flow.ExpandedIdleRatePerSec, 0.0001f,
        "promotion = new-gameplay ticket: idle output expanded with the halls");
    var row = G18Catalogs.RankById(G18Catalogs.LoadRanks(), "rank_2");
    Assert.AreEqual(row.nameKey, flow.CeremoniedTitleKey, "title key = frozen ladder nameKey (U129 gate)");
    // parity: a second presenter over the same sim reads identical state
    var flowB = new RankUpFlow(sim);
    Assert.AreEqual(flow.NextCeremonyRank, flowB.NextCeremonyRank);
    Assert.AreEqual(flow.CeremoniedTitleKey, flowB.CeremoniedTitleKey);
    Assert.AreEqual(flow.CeremoniedBadgeTier, flowB.CeremoniedBadgeTier);
    Assert.AreEqual(flow.PendingCount, flowB.PendingCount);
    // drain once: due clears, a second present is a no-op, zero re-roll
    flow.MarkPresented();
    Assert.AreEqual(1, flow.PresentedCount);
    Assert.IsFalse(flow.CeremonyDue);
    flow.MarkPresented();
    Assert.AreEqual(1, flow.PresentedCount, "a presented promotion never re-triggers");
    Assert.AreEqual(1, sim.CountEvents("rank_up"), "the presenter never emits or edits sim events");
    Assert.IsTrue(flowB.CeremonyDue, "each wall drains its own presented counter");
}
[Test]
```

```
        public void StackedPromotions_DrainOneCeremonyAtATime_LadderTopsOut()
        {
            var sim = new PalaceSim(31);
            var flow = new RankUpFlow(sim);
            for (int i = 0; i &lt; 6; i++)
            {
                Fund(sim);
                Assert.IsTrue(sim.BuildHall());
            }
            Assert.AreEqual(2, sim.CountEvents("rank_up"), "six halls = two stacked promotions");
            Assert.AreEqual(2, flow.PendingCount, "stacked over a long offline window");
            Assert.AreEqual(2, flow.NextCeremonyRank, "drains the oldest promotion first");
            flow.MarkPresented();
            Assert.AreEqual(1, flow.PendingCount);
            Assert.AreEqual(3, flow.NextCeremonyRank, "the second ceremony celebrates rank 3");
            Assert.AreEqual("A04", flow.CeremoniedBadgeTier);
            flow.MarkPresented();
            Assert.IsFalse(flow.CeremonyDue);
            Assert.AreEqual(2, flow.PresentedCount);
            // complete the palace: the ladder tops out, no sixth rank exists
            for (int i = 0; i &lt; 6; i++)
            {
                Fund(sim);
                Assert.IsTrue(sim.BuildHall());
            }
            Assert.AreEqual(PalaceSim.HallSlotCap, sim.Halls);
            Assert.AreEqual(5, sim.Rank);
            Assert.AreEqual(4, flow.RankUpCount);
            flow.MarkPresented();
            flow.MarkPresented();
            Assert.AreEqual(4, flow.PresentedCount, "presented never outruns the emitted events");
            Assert.IsFalse(flow.CeremonyDue, "ladder complete: no fifth promotion can stack");
        }
    }
}
// PalaceRevengeFlowTests.cs
using System;
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S1-T005 slice-a acceptance (taskbook spec block: EM trio = revenge queue
    // determinism inside the locked 4-10s window / share-card data ports readable
    // in both report states / album entry + progress anchor). RaidFlow test
    // paradigm: the raid verdict rides the SetHaulChance seam (1.0/0.0) where the
    // raid ring feeds the faces; the revenge verdict rides the sim's own 0.80
    // roll, so the readouts are checked against whichever state fired (verdict
    // membership + purse parity), keeping the trajectories seed-independent.
    public class PalaceRevengeFlowTests
    {
```

```
static void GrantOneTicket(PalaceSim sim)
{
    sim.ForceNextSpin(PalaceSim.WheelOutcome.RaidTicket);
    Assert.IsTrue(sim.SpinWheel(), "forced ticket spin must land");
}
static PalaceSim RobbedSim(int seed)
{
    var sim = new PalaceSim(seed);
    GrantOneTicket(sim);
    sim.SetHaulChance(1f);
    Assert.IsTrue(sim.RaidOpponent(), "hauling raid arms the retaliation queue");
    sim.AdvanceClock(20f);
    Assert.IsTrue(sim.RevengePending, "robbery arms the revenge seat");
    return sim;
}
[Test]
public void RevengeQueue_Window_Determinism_And_Consumption()
{
    // locked band face (numeric pack section 2, E4 measured 4-10s)
    Assert.AreEqual(4f, RevengeFlow.RevengeWindowMinSec);
    Assert.AreEqual(10f, RevengeFlow.RevengeWindowMaxSec);
    // every roster seat x a count sweep stays inside the band, deterministically
    for (int i = 0; i &lt; PalaceSim.OpponentRoster.Length; i++)
    {
        for (int c = 0; c &lt; 4; c++)
        {
            string t = PalaceSim.OpponentRoster[i];
            float w = RevengeFlow.RevengeWindowSecOf(t, c);
            Assert.GreaterOrEqual(w, RevengeFlow.RevengeWindowMinSec);
            Assert.LessOrEqual(w, RevengeFlow.RevengeWindowMaxSec);
            Assert.AreEqual(w, RevengeFlow.RevengeWindowSecOf(t, c),
                "same inputs -&gt; same window (seed determinism, presenter never re-rolls)");
        }
    }
    // live queue readout: the armed seat exposes code + target + window
    var sim = RobbedSim(2024);
    var flow = new RevengeFlow(sim);
    Assert.IsTrue(flow.Pending);
    Assert.AreEqual(sim.RevengeTarget, flow.PendingTarget);
    Assert.AreEqual(PalaceSim.RevengeCodeOf(flow.PendingTarget), flow.PendingCode,
        "code parsed off the revenge_code event, zero re-roll");
    Assert.GreaterOrEqual(flow.QueueWindowSec, RevengeFlow.RevengeWindowMinSec);
    Assert.LessOrEqual(flow.QueueWindowSec, RevengeFlow.RevengeWindowMaxSec);
    // bad code rides the sim reject stream, the seat stays armed
    Assert.IsFalse(flow.EnterCode("FRM-Nobody-0000"));
    Assert.AreEqual(1, sim.CountEvents("op_rejected"));
    Assert.IsTrue(flow.Pending);
    // accept = the revenge raid launches: verdict readouts + purse parity
    int before = sim.Coins;
    Assert.IsTrue(flow.EnterCode(flow.PendingCode));
```

```
        Assert.IsFalse(flow.Pending, "consumed seat clears");
        Assert.IsTrue(flow.LastVerdict == "haul" || flow.LastVerdict == "countersteal");
        Assert.AreEqual(flow.LastTarget, sim.RevengeTarget, "report carries the robbed-by face");
        Assert.Greater(flow.LastComedyClause.Length, 0, "comedy clause token rides the report");
        if (flow.LastVerdict == "haul")
        {
            Assert.Greater(flow.LastLoot, 0);
            Assert.AreEqual(0, flow.LastLoss);
            Assert.AreEqual(before + flow.LastLoot, sim.Coins, "haul lands in the purse");
        }
        else
        {
            Assert.AreEqual(5, flow.LastLoss, "revenge counter-steal reads the fixed 5 loss");
            Assert.AreEqual(0, flow.LastLoot);
            Assert.AreEqual(before - flow.LastLoss, sim.Coins, "loss leaves the purse");
        }
        Assert.AreEqual(flow.LastVerdict == "haul" ? flow.LastLoot : 0, flow.RevengeLootLedger,
            "loot ledger parity");
        Assert.AreEqual(flow.LastVerdict == "countersteal" ? flow.LastLoss : 0, flow.RevengeLossLedger,
            "loss ledger parity");
        // revenge share port: deterministic second-round share material
        string share = flow.RevengeShareCode;
        Assert.IsTrue(share.StartsWith("PLN-"), "X032 offline share-code form");
        Assert.AreEqual(share, flow.RevengeShareCode, "pure derivation, zero re-rolls");
        // consumed seat: further codes ride the no-pending reject
        Assert.IsFalse(flow.EnterCode(flow.PendingCode));
        Assert.AreEqual(2, sim.CountEvents("op_rejected"));
    }
    [Test]
    public void ShareCard_Ports_Both_Report_States_Readable()
    {
        // haul state: the winning report carries a share code
        var simA = new PalaceSim(777);
        GrantOneTicket(simA);
        simA.SetHaulChance(1f);
        var flowA = new RevengeFlow(simA);
        Assert.IsTrue(simA.RaidOpponent());
        Assert.Greater(flowA.LastRaidShareCode.Length, 0, "haul report carries a share code");
        Assert.AreEqual(1, flowA.ShareCodeObserved);
        // countersteal state: the losing report is equally shareable
        var simB = new PalaceSim(31);
        GrantOneTicket(simB);
        simB.SetHaulChance(0f);
        var flowB = new RevengeFlow(simB);
        Assert.IsTrue(simB.RaidOpponent());
        Assert.Greater(flowB.LastRaidShareCode.Length, 0,
            "loss reports share too (win or loss, always shareable)");
        Assert.AreEqual(1, flowB.ShareCodeObserved);
        Assert.AreEqual(1, simB.CountEvents("raid_report"));
    }
```

```
        [Test]
        public void AlbumEntry_Progress_Anchor_Decrements_Stays_In_Band()
        {
            var sim = new PalaceSim(55);
            var album = new AlbumFlow(sim);
            Assert.AreEqual(0, album.Collected);
            Assert.AreEqual(G18Catalogs.AlbumItemCount, album.TotalItems, "locked 24-item face");
            Assert.AreEqual(G18Catalogs.AlbumItemCount, album.RemainingToComplete,
                "empty album: distance = full face");
            // first haul lands a curio: the anchor decrements by exactly one
            GrantOneTicket(sim);
            sim.SetHaulChance(1f);
            Assert.IsTrue(sim.RaidOpponent());
            Assert.AreEqual(1, album.GalleryAddObserved);
            Assert.AreEqual(1, album.Collected);
            Assert.AreEqual(G18Catalogs.AlbumItemCount - 1, album.RemainingToComplete,
                "entry decrements the distance by one");
            string curio = album.LastCurio;
            Assert.IsTrue(curio.StartsWith("CR-"), "sim curio id form CR-target-tier");
            string tier = AlbumFlow.TierOfCurio(curio);
            Assert.IsTrue(tier == G18Catalogs.TierBronze || tier == G18Catalogs.TierSilver
                || tier == G18Catalogs.TierGold || tier == G18Catalogs.TierJade,
                "four-tier attribution off the tier digit");
            // id-space note (C986 precedent): sim curio ids and album table row
            // ids (B/S/G/J) are different spaces - the catalog law answers null
            // for sim curios; wall-grid row consumption is the slice-b seat.
            Assert.IsNull(album.RowOf(curio), "catalog law: unknown id reads null");
            // day rollover rotates the opponent -&gt; the second haul lands a
            // distinct target curio regardless of the tier roll (deterministic)
            sim.AdvanceClock(86500f);
            GrantOneTicket(sim);
            Assert.IsTrue(sim.RaidOpponent(), "day-1 raid against the rotated opponent");
            Assert.AreEqual(2, album.Collected, "distinct target = distinct curio id");
            Assert.AreEqual(G18Catalogs.AlbumItemCount - 2, album.RemainingToComplete,
                "second distinct curio decrements again");
            Assert.GreaterOrEqual(album.RemainingToComplete, 0, "anchor never leaves the band");
            Assert.LessOrEqual(album.RemainingToComplete, G18Catalogs.AlbumItemCount);
            Assert.AreNotEqual(curio, album.LastCurio, "rotated opponent lands a different curio");
        }
    }
}
// PalaceWheelFlowTests.cs
using System.Collections.Generic;
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S1-T002 slice-a acceptance (G12BoardPresenter paradigm, zero PalaceSim
    // changes): the wheel presenter owns the session rules the sim does not
    // - free spin zero stamina, natural cap 50/day, rewarded +10 grants at
```

```
// most twice a day, day-rollover reset - and stays in literal parity
// with the sim event stream (outcome ids read from the stream, payouts
// read as resource deltas, never re-rolled). H2 first-operation target:
// the wheel is ready at t=0 so the first tap always lands.
public class PalaceWheelFlowTests
{
    [Test]
    public void First_Operation_Wheel_Ready_At_T0_And_First_Spin_Lands_Coins()
    {
        var sim = new PalaceSim(12345);
        sim.Open();
        var flow = new WheelFlow(sim);
        Assert.IsTrue(flow.CanSpinNow, "H2 target: the first operation must be available at t=0");
        Assert.AreEqual(0, flow.SpinsDone);
        int coinsBefore = sim.Coins;
        Assert.IsTrue(flow.Spin(), "the first tap must always land (free spin, zero stamina)");
        Assert.AreEqual(1, flow.SpinsDone);
        Assert.AreEqual(G18Catalogs.WheelCoins, flow.LastOutcomeId,
            "Open() forces the first spin to Coins (E4 H3 construction)");
        Assert.AreEqual(sim.Coins - coinsBefore, flow.LastPayout,
            "payout parity: the presenter reads the resource delta");
        Assert.GreaterOrEqual(flow.LastPayout, 8, "coins payout stays in the E4 band 8..15");
        Assert.LessOrEqual(flow.LastPayout, 15, "coins payout stays in the E4 band 8..15");
    }
    [Test]
    public void Free_Spin_Never_Consumes_Stamina_Or_Purse()
    {
        var sim = new PalaceSim(2024);
        var flow = new WheelFlow(sim);
        sim.ForceNextSpin(PalaceSim.WheelOutcome.Timber);
        int coinsBefore = sim.Coins;
        Assert.IsTrue(flow.Spin());
        Assert.AreEqual(coinsBefore, sim.Coins,
            "free spins must never spend coins (zero-stamina, frozen at admission)");
        Assert.Greater(sim.Timber, 0, "a timber spin must actually pay timber");
        Assert.AreEqual(0, sim.CountEvents("op_rejected"),
            "spinning inside the cap must never emit a rejection");
    }
    [Test]
    public void Natural_Cap_50_Blocks_The_51st_Spin_Without_Touching_The_Sim()
    {
        var sim = new PalaceSim(7);
        var flow = new WheelFlow(sim);
        for (int i = 0; i &lt; WheelFlow.NaturalSpinsPerDay; i++)
        {
            Assert.IsTrue(flow.Spin(), "natural spin #" + i + " must land");
        }
        Assert.AreEqual(0, flow.NaturalSpinsLeftToday);
        Assert.AreEqual(WheelFlow.NaturalSpinsPerDay, sim.SpinCount);
        Assert.IsFalse(flow.Spin(), "the 51st free spin of the day must be blocked");
```

```
        Assert.AreEqual(WheelFlow.NaturalSpinsPerDay, sim.SpinCount,
            "a blocked spin must never reach the sim");
        Assert.IsFalse(flow.CanSpinNow);
    }
    [Test]
    public void Rewarded_Grant_Plus_Ten_Caps_At_Two_Per_Day_No_Side_Effects()
    {
        var sim = new PalaceSim(5);
        var flow = new WheelFlow(sim);
        for (int i = 0; i &lt; 50; i++) flow.Spin();
        Assert.IsFalse(flow.CanSpinNow, "natural pool exhausted first");
        Assert.IsTrue(flow.GrantBonusSpins());
        Assert.AreEqual(10, flow.BonusSpins);
        Assert.AreEqual(1, flow.BonusGrantsToday);
        for (int i = 0; i &lt; 10; i++)
        {
            Assert.IsTrue(flow.Spin(), "bonus spin #" + i + " must land");
        }
        Assert.IsFalse(flow.Spin(), "bonus pool exhausted again");
        Assert.IsTrue(flow.GrantBonusSpins(), "a second grant of the same day is allowed");
        Assert.AreEqual(10, flow.BonusSpins);
        for (int i = 0; i &lt; 10; i++)
        {
            Assert.IsTrue(flow.Spin(), "second-grant bonus spin #" + i + " must land");
        }
        Assert.IsFalse(flow.GrantBonusSpins(), "a third grant of the same day must be denied");
        Assert.AreEqual(0, flow.BonusSpins, "a denied grant must not change the pool");
        Assert.AreEqual(2, flow.BonusGrantsToday);
        Assert.AreEqual(70, sim.SpinCount, "50 natural + 20 bonus spins reached the sim");
    }
    [Test]
    public void Day_Rollover_Resets_Every_Cap_And_Wipes_Unspent_Bonus()
    {
        var sim = new PalaceSim(11);
        var flow = new WheelFlow(sim);
        for (int i = 0; i &lt; 50; i++) flow.Spin();
        Assert.IsTrue(flow.GrantBonusSpins());
        for (int i = 0; i &lt; 5; i++) flow.Spin(); // spend half the bonus pool
        Assert.AreEqual(5, flow.BonusSpins);
        sim.AdvanceClock(86400f); // one full day later, still deterministic
        Assert.AreEqual(50, flow.NaturalSpinsLeftToday, "a new day refills the natural cap");
        Assert.AreEqual(0, flow.BonusGrantsToday, "a new day reopens the grant seat");
        Assert.AreEqual(0, flow.BonusSpins, "unspent bonus spins must not survive the day line");
        Assert.IsTrue(flow.CanSpinNow);
    }
    [Test]
    public void Outcome_Payout_Parity_Across_All_Four_Faces()
    {
        var sim = new PalaceSim(2026);
        var flow = new WheelFlow(sim);
```

```
    var faces = new[]
    {
        PalaceSim.WheelOutcome.Coins,
        PalaceSim.WheelOutcome.Timber,
        PalaceSim.WheelOutcome.EventCard,
        PalaceSim.WheelOutcome.RaidTicket
    };
    var expectedIds = new[]
    {
        G18Catalogs.WheelCoins,
        G18Catalogs.WheelTimber,
        G18Catalogs.WheelEventCard,
        G18Catalogs.WheelRaidTicket
    };
    for (int f = 0; f &lt; faces.Length; f++)
    {
        sim.ForceNextSpin(faces[f]);
        int coinsBefore = sim.Coins;
        int timberBefore = sim.Timber;
        int ticketsBefore = sim.RaidTickets;
        Assert.IsTrue(flow.Spin());
        Assert.AreEqual(expectedIds[f], flow.LastOutcomeId,
            "outcome id must mirror the cfg_g18_wheel table row");
        switch (expectedIds[f])
        {
            case G18Catalogs.WheelCoins:
            case G18Catalogs.WheelEventCard:
                Assert.AreEqual(sim.Coins - coinsBefore, flow.LastPayout,
                    "coins-family payout parity = resource delta");
                break;
            case G18Catalogs.WheelTimber:
                Assert.AreEqual(sim.Timber - timberBefore, flow.LastPayout,
                    "timber payout parity = resource delta");
                break;
            default:
                Assert.AreEqual(sim.RaidTickets - ticketsBefore, flow.LastPayout,
                    "ticket payout parity = resource delta");
                Assert.AreEqual(1, flow.LastPayout, "a raid-ticket spin pays exactly one ticket");
                break;
        }
    }
}
[Test]
public void Flow_Faces_Cover_Exactly_The_Frozen_Wheel_Table()
{
    var wheel = G18Catalogs.LoadWheel();
    Assert.IsNotNull(wheel, "cfg_g18_wheel.json must load");
    var sim = new PalaceSim(31);
    var flow = new WheelFlow(sim);
    var seen = new HashSet&lt;string&gt;();
```

```
            var all = new[]
            {
                PalaceSim.WheelOutcome.Coins,
                PalaceSim.WheelOutcome.Timber,
                PalaceSim.WheelOutcome.EventCard,
                PalaceSim.WheelOutcome.RaidTicket
            };
            for (int f = 0; f &lt; all.Length; f++)
            {
                sim.ForceNextSpin(all[f]);
                Assert.IsTrue(flow.Spin());
                Assert.IsTrue(seen.Add(flow.LastOutcomeId),
                    "each wheel face must map to a distinct table outcome id");
            }
            for (int i = 0; i &lt; wheel.segments.Length; i++)
            {
                Assert.IsTrue(seen.Contains(wheel.segments[i].outcome),
                    "flow must expose every table outcome id: " + wheel.segments[i].outcome);
            }
        }
    }
}
// RaidViewHostTests.cs
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S1-T004 slice b acceptance, pure-mapping faces (WheelViewHostTests
    // paradigm): the view owns ZERO invented numbers or comedy text - every
    // verdict, comedy token, roster seat and delta maps through a pure static
    // into an i18n key, and unknown tokens fall back to honest raw faces
    // (empty key -&gt; raw token shown, never invented copy).
    public class RaidViewHostTests
    {
        [Test]
        public void ComedyTitleKeyOf_MapsVerdicts_UnknownFallsBackNeutral()
        {
            Assert.AreEqual("g18.raid.report.haul", RaidViewHost.ComedyTitleKeyOf("haul"));
            Assert.AreEqual("g18.raid.report.countersteal", RaidViewHost.ComedyTitleKeyOf("countersteal"));
            Assert.AreEqual("g18.raid.report.title", RaidViewHost.ComedyTitleKeyOf(""),
                "unknown verdict falls back to the neutral report title");
            Assert.AreEqual("g18.raid.report.title", RaidViewHost.ComedyTitleKeyOf("nonsense"));
        }
        [Test]
        public void ComedyClauseKeyOf_MapsKnownTokens_UnknownFallsBackEmpty()
        {
            Assert.AreEqual("g18.raid.comedy.haul",
                RaidViewHost.ComedyClauseKeyOf("guard-ate-his-own-bait"),
                "the sim's haul comedy clause token maps to its i18n key");
            Assert.AreEqual("g18.raid.comedy.countersteal",
```

```
        RaidViewHost.ComedyClauseKeyOf("pie-in-the-face-pivot"),
        "the sim's countersteal comedy clause token maps to its i18n key");
    Assert.AreEqual("", RaidViewHost.ComedyClauseKeyOf("unknown-token"),
        "unknown tokens map to empty -&gt; the view shows the raw token, never invented comedy");
}
[Test]
public void DeltaTextOf_SignsGainsAndLosses()
{
    Assert.AreEqual("+37", RaidViewHost.DeltaTextOf(true, 37), "haul delta reads as a gain");
    Assert.AreEqual("-8", RaidViewHost.DeltaTextOf(false, 8), "countersteal delta reads as a loss");
}
[Test]
public void OpponentNameKeyOf_MapsAllSixSimSeats_UnknownFallsBackEmpty()
{
    // the runtime roster is the sim face (six fiction seats, C986
    // bookkeeping note: cfg table carries eight; the view follows the sim)
    Assert.AreEqual("g18.roster.r01.name", RaidViewHost.OpponentNameKeyOf("MarquisMiser"));
    Assert.AreEqual("g18.roster.r02.name", RaidViewHost.OpponentNameKeyOf("DukeGlutton"));
    Assert.AreEqual("g18.roster.r03.name", RaidViewHost.OpponentNameKeyOf("CountessNap"));
    Assert.AreEqual("g18.roster.r04.name", RaidViewHost.OpponentNameKeyOf("BaronBluff"));
    Assert.AreEqual("g18.roster.r05.name", RaidViewHost.OpponentNameKeyOf("ViscountVain"));
    Assert.AreEqual("g18.roster.r06.name", RaidViewHost.OpponentNameKeyOf("LadyLantern"));
    Assert.AreEqual("", RaidViewHost.OpponentNameKeyOf("MasterAbacus"),
        "cfg-only seats (r07/r08) are NOT mapped on the sim face - no invented sim seats");
}
[Test]
public void RaidI18nKeys_AllPresentAfterLoad()
{
    G18Text.Load();
    string[] keys =
    {
        "g18.raid.tab", "g18.raid.title", "g18.raid.opponent", "g18.raid.tickets",
        "g18.raid.launch.button", "g18.raid.noticket", "g18.raid.report.title",
        "g18.raid.report.haul", "g18.raid.report.countersteal", "g18.raid.report.share",
        "g18.raid.comedy.haul", "g18.raid.comedy.countersteal",
        "g18.raid.double.button", "g18.raid.double.pending",
        "g18.raid.double.denied.cap", "g18.raid.double.denied.second",
        "g18.raid.rescue.title", "g18.raid.rescue.body", "g18.raid.rescue.accept",
        "g18.raid.rescue.decline", "g18.raid.rescue.restored", "g18.raid.rescue.declined",
        "g18.raid.close",
    };
    foreach (string k in keys)
    {
        Assert.IsTrue(G18Text.Has(k), "raid view key must be declared and loaded: " + k);
    }
}
// ---- S3-T003 M08: battle-report share card (loot slot + share op) ----
[Test]
public void LootSpotKeyOf_MapsBothVerdictStates_BothStayShareable()
{
```

```
            Assert.AreEqual("g18.raid.share.curio", RaidViewHost.LootSpotKeyOf(true),
                "a haul shows the looted curio on the share card");
            Assert.AreEqual("g18.raid.share.emptyhands", RaidViewHost.LootSpotKeyOf(false),
                "a countersteal shows the honest empty-hands face - the loss state stays shareable");
        }
        [Test]
        public void LootFaceOf_JoinsCurioWithTierName_EmptyStaysEmpty()
        {
            Assert.AreEqual("CR-MarquisMiser-1 - Bronze", RaidViewHost.LootFaceOf("CR-MarquisMiser-1", "Bronze"),
                "the loot slot joins the sim curio id with its catalog tier name");
            Assert.AreEqual("CR-DukeGlutton-2", RaidViewHost.LootFaceOf("CR-DukeGlutton-2", ""),
                "a missing tier name falls back to the raw curio id, never invented text");
            Assert.AreEqual("", RaidViewHost.LootFaceOf("", "Bronze"),
                "an empty curio (countersteal) stays empty - the slot never invents a trophy");
            Assert.AreEqual("", RaidViewHost.LootFaceOf(null, "Bronze"));
        }
        [Test]
        public void RaidShareI18nKeys_AllPresentAfterLoad()
        {
            G18Text.Load();
            string[] keys =
            {
                "g18.raid.share.curio", "g18.raid.share.emptyhands",
                "g18.raid.share.button", "g18.raid.share.staged",
            };
            foreach (string k in keys)
            {
                Assert.IsTrue(G18Text.Has(k), "M08 share-card key must be declared and loaded: " + k);
            }
        }
        [Test]
        public void ReportBeat_Maps_Each_Verdict_To_Its_Own_Voice()
        {
            // S3-T004: the battle report lands with its verdict's own beat -
            // a haul flourishes, a countersteal sags comedically, and any
            // other verdict stays silent (the presenter never invents).
            Assert.AreEqual(G18Sfx.RaidHaul, RaidViewHost.ReportBeatOf("haul"),
                "a haul lands the ascending spoils flourish");
            Assert.AreEqual(G18Sfx.RaidCounter, RaidViewHost.ReportBeatOf("countersteal"),
                "a countersteal lands the comedic gong sag");
            Assert.AreEqual("", RaidViewHost.ReportBeatOf("mystery"),
                "unknown verdicts stay silent");
            Assert.AreEqual("", RaidViewHost.ReportBeatOf(""),
                "empty verdicts stay silent");
            Assert.AreEqual("", RaidViewHost.ReportBeatOf(null),
                "null verdicts stay silent");
        }
    }
}
// RevengeViewHostTests.cs
```

```
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S1-T005 slice b acceptance, pure-mapping faces (RaidViewHostTests
    // paradigm): the view owns ZERO invented numbers or comedy text - every
    // verdict and comedy token maps through a pure static into an i18n key,
    // and unknown tokens fall back to honest raw faces (empty key -&gt; raw
    // token shown, never invented copy).
    public class RevengeViewHostTests
    {
        [Test]
        public void ComedyTitleKeyOf_MapsVerdicts_UnknownFallsBackNeutral()
        {
            Assert.AreEqual("g18.revenge.report.haul", RevengeViewHost.ComedyTitleKeyOf("haul"));
            Assert.AreEqual("g18.revenge.report.countersteal", RevengeViewHost.ComedyTitleKeyOf("countersteal"));
            Assert.AreEqual("g18.revenge.report.title", RevengeViewHost.ComedyTitleKeyOf(""),
                "unknown verdict falls back to the neutral report title");
            Assert.AreEqual("g18.revenge.report.title", RevengeViewHost.ComedyTitleKeyOf("nonsense"));
        }
        [Test]
        public void ComedyClauseKeyOf_MapsKnownTokens_UnknownFallsBackEmpty()
        {
            Assert.AreEqual("g18.revenge.comedy.haul",
                RevengeViewHost.ComedyClauseKeyOf("knew-the-doghole"),
                "the sim's revenge haul comedy clause token maps to its i18n key");
            Assert.AreEqual("g18.revenge.comedy.countersteal",
                RevengeViewHost.ComedyClauseKeyOf("they-moved-the-vault"),
                "the sim's revenge countersteal comedy clause token maps to its i18n key");
            Assert.AreEqual("", RevengeViewHost.ComedyClauseKeyOf("unknown-token"),
                "unknown tokens map to empty -&gt; the view shows the raw token, never invented comedy");
        }
        [Test]
        public void DeltaTextOf_SignsGainsAndLosses()
        {
            Assert.AreEqual("+12", RevengeViewHost.DeltaTextOf(true, 12), "revenge haul delta reads as a gain");
            Assert.AreEqual("-5", RevengeViewHost.DeltaTextOf(false, 5), "revenge countersteal delta reads as a loss");
        }
        [Test]
        public void RevengeI18nKeys_AllPresentAfterLoad()
        {
            G18Text.Load();
            string[] keys =
            {
                "g18.revenge.tab", "g18.revenge.title",
                "g18.revenge.pending.title", "g18.revenge.pending.code", "g18.revenge.pending.window",
                "g18.revenge.enter.button",
                "g18.revenge.report.title", "g18.revenge.report.haul", "g18.revenge.report.countersteal",
                "g18.revenge.comedy.haul", "g18.revenge.comedy.countersteal",
                "g18.revenge.report.share", "g18.revenge.share.staged", "g18.revenge.share.button",
```

```
                "g18.revenge.close",
            };
            foreach (string k in keys)
            {
                Assert.IsTrue(G18Text.Has(k), "revenge view key must be declared and loaded: " + k);
            }
        }
    }
}
// WheelViewHostTests.cs
using NUnit.Framework;
using Sy.Palace;
namespace Sy.Palace.Tests
{
    // S1-T002 slice-b view parity (G17 PlayMode smoke carries the scene-walk
    // face; this EditMode suite pins the deterministic spin geometry the
    // animation rides on - landing math, tick counting, pitch ladder, easing
    // and the zero-file tick synthesis). All faces are pure statics: the
    // asserts stay frame-rate independent by construction.
    public class WheelViewHostTests
    {
        [Test]
        public void End_Rotation_Lands_Its_Outcome_Segment_Under_The_Pointer()
        {
            for (int outcome = 0; outcome &lt; 4; outcome++)
            {
                float start = 37f;
                float end = WheelViewHost.EndRotationFor(outcome, start);
                Assert.AreEqual(outcome, WheelViewHost.SegmentAtPointer(end),
                    "outcome " + outcome + " must center under the pointer at the sweep end");
                Assert.Greater(end - start, WheelViewHost.SpinMinTurnsDeg,
                    "the sweep must carry at least the minimum three turns");
            }
        }
        [Test]
        public void End_Rotation_Works_From_Any_Current_Rotation_Including_Multi_Turn()
        {
            float[] starts = { 0f, 89f, 360f, 721.5f, -45f };
            for (int s = 0; s &lt; starts.Length; s++)
            {
                float end = WheelViewHost.EndRotationFor(2, starts[s]);
                Assert.AreEqual(2, WheelViewHost.SegmentAtPointer(end),
                    "from start " + starts[s] + " the event_card segment must still land centered");
            }
        }
        [Test]
        public void Tick_Count_Equals_Segment_Boundaries_Crossed_And_Stay_In_The_Sweep_Band()
        {
            float start = 123f;
            float end = WheelViewHost.EndRotationFor(3, start);
```

```
            int ticks = WheelViewHost.TickCountBetween(start, end);
            Assert.GreaterOrEqual(ticks, 12, "three full turns cross at least 12 boundaries");
            Assert.LessOrEqual(ticks, 16, "one extra alignment turn keeps the ladder under 16 ticks");
            Assert.AreEqual(ticks, (int)(System.Math.Floor(end / 90.0) - System.Math.Floor(start / 90.0)),
                "tick counting must stay the boundary-floor law (frame-rate independent)");
        }
        [Test]
        public void Tick_Pitch_Ladder_Walks_One_Plus_Two_Percent_Per_Tick()
        {
            for (int i = 0; i &lt; 16; i++)
            {
                Assert.AreEqual(1.00f + 0.02f * i, WheelViewHost.PitchOfTick(i), 0.0001f,
                    "pitch ladder tick " + i + " must hold the 1.00 + 0.02 * i feel anchor");
            }
        }
        [Test]
        public void Ease_Out_Cubic_Starts_At_Zero_Ends_At_One_And_Never_Overshoots()
        {
            Assert.AreEqual(0f, WheelViewHost.EaseOutCubic(0f));
            Assert.AreEqual(1f, WheelViewHost.EaseOutCubic(1f));
            float prev = 0f;
            for (int i = 1; i &lt;= 20; i++)
            {
                float u = i / 20f;
                float v = WheelViewHost.EaseOutCubic(u);
                Assert.GreaterOrEqual(v, prev, "the ease must be monotonic (never backtrack)");
                Assert.LessOrEqual(v, 1f, "the ease must never overshoot the end rotation");
                prev = v;
            }
        }
        [Test]
        public void Spin_Tick_Synthesis_Is_Deterministic_And_Caliber_Capped()
        {
            var a = G18Sfx.Synth(G18Sfx.SpinTick);
            var b = G18Sfx.Synth(G18Sfx.SpinTick);
            Assert.AreEqual(a.Length, b.Length, "the tick render must be deterministic");
            Assert.Greater(a.Length, 0, "the tick must render actual samples");
            float peak = 0f;
            for (int i = 0; i &lt; a.Length; i++)
            {
                Assert.AreEqual(a[i], b[i], "sample " + i + " must be bit-identical across renders");
                float abs = a[i] &lt; 0f ? -a[i] : a[i];
                if (abs &gt; peak) peak = abs;
            }
            Assert.Greater(peak, 0.05f, "the tick must be audible, not silence");
            Assert.LessOrEqual((double)peak, G18Sfx.SamplePeakCap,
                "sample peak must keep the -3.6 dBFS caliber cap (true-peak margin law)");
        }
    }
}
```